



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Компьютерные системы и сети

НАПРАВЛЕНИЕ ПОДГОТОВКИ 09.03.01/03 Вычислительные машины, комплексы,
системы и сети

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ БАКАЛАВРА НА ТЕМУ:

*Программно-аппаратная система
автопилотирования автомобилей в городской среде*

Студент

ИУ6-83Б

(Группа)

(Подпись, дата)

М.Е. Дорошин

(И.О. Фамилия)

Руководитель ВКРБ

(Подпись, дата)

Е.Ю. Оболенская

(И.О. Фамилия)

Нормоконтролер

(Подпись, дата)

(И.О. Фамилия)

**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

УТВЕРЖДАЮ

Заведующий кафедрой ИУ6

_____ А.В. Пролетарский
« » _____ 2026 г.

З А Д А Н И Е
на выполнение выпускной квалификационной работы бакалавра

Студент группы ИУ6-83Б

_____ Дорошин Максим Евгеньевич

(Фамилия, имя, отчество)

Тема квалификационной работы: Программно-аппаратная система автопилотирования
автомобилей в городской среде

При выполнении ВКР:

Используются / Не используются	Да/Нет
1) Литературные источники и документы, имеющие гриф секретности	Нет
2) Литературные источники и документы, имеющие пометку «Для служебного пользования», иных пометок, запрещающих открытое опубликование	Нет
3) Служебные материалы других организаций	Нет
4) Результаты НИР (ОКР), выполняемой в МГТУ им. Н.Э. Баумана	Нет
5) Материалы по незавершенным исследованиям или материалы по завершенным исследованиям, но ещё не опубликованные в открытой печати	Нет

Тема квалификационной работы утверждена распоряжением по факультету:	
Название факультета:	Информатика и системы управления
Дата и рег. номер распоряжения:	№ 30.30-30/1781 от 28.11.2025 г.

Часть 1. Исследовательская

Провести аналитический обзор различных систем автопилотирования и сравнить их особенности.

Часть 2. Конструкторская

Осуществить выбор технологии и средств разработки для программного и аппаратного обеспечения. Разработать структуру программного и аппаратного обеспечения и определить их компоненты. Выполнить проектирование программных и аппаратных компонентов.

Реализовать компоненты аппаратного и программного обеспечения. Выполнить сборку аппаратного и программного обеспечения и произвести их тестирование.

Часть 3. Технологическая

Разработать технологию удалённого обновления прошивки микроконтроллера. Разработать технологию комплексного тестирования системы в различных условиях.

Оформление квалификационной работы:

Расчетно-пояснительная записка на 55–65 листах формата А4.

Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.)

1. Схема функциональная программной части.

2. Схема функциональная макета автомобиля.

3. Схема принципиальная макета автомобиля.

4. Фотографии макета системы.

5. Формы веб-интерфейса.

6. Результаты анализа систем автопилотирования.

Дата выдачи задания « 1 » сентября 2025 г.

В соответствии с учебным планом выпускную квалификационную работу выполнить в полном объеме в срок до « 1 » июня 2026 г.

**Руководитель квалификационной
работы**

(Подпись, дата)

Е.Ю. Оболенская

(И.О. Фамилия)

Студент

(Подпись, дата)

М.Е. Дорошин

(И.О. Фамилия)

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

УТВЕРЖДАЮ

КАФЕДРА Компьютерные системы и сети

Заведующий кафедрой ИУ6

ГРУППА ИУ6-83Б

_____ А.В. Пролетарский
« _____ » _____ 2026 г.

КАЛЕНДАРНЫЙ ПЛАН

выполнения выпускной квалификационной работы бакалавра

студента: _____ **Дорошина Максима Евгеньевича**

(фамилия, имя, отчество)

Тема квалификационной работы: Программно-аппаратная система автопилотирования
автомобилей в городской среде

№ п/п	Наименование этапов выпускной квалификационной работы	Сроки выполнения этапов		Отметка о выполнении	
		план	факт	Должность	ФИО, подпись
1.	Задание на выполнение работы. Формулирование проблемы, цели и задач работы	<u>09.2025</u> <i>Планируемая дата</i>		Руководитель ВКР	Е.Ю. Оболенская
2.	1 часть <u>Исследовательская</u>	<u>12.2025</u> <i>Планируемая дата</i>		Руководитель ВКР	Е.Ю. Оболенская
3.	Утверждение окончательных формулировок решаемой проблемы, цели работы и перечня задач	<u>02.2026</u> <i>Планируемая дата</i>		Заведующий кафедрой	А.В. Пролетарский
4.	2 часть <u>Конструкторская</u>	<u>05.2026</u> <i>Планируемая дата</i>		Руководитель ВКР	Е.Ю. Оболенская
5.	3 часть <u>Технологическая</u>	<u>05.2026</u> <i>Планируемая дата</i>		Руководитель ВКР	Е.Ю. Оболенская
6.	1-я редакция работы	<u>05.2026</u> <i>Планируемая дата</i>		Руководитель ВКР	Е.Ю. Оболенская
7.	Подготовка доклада и презентации	<u>06.2026</u> <i>Планируемая дата</i>		Руководитель ВКР	Е.Ю. Оболенская
8.	Отзыв руководителя	<u>06.2026</u> <i>Планируемая дата</i>		Руководитель ВКР	Е.Ю. Оболенская
9.	Нормоконтроль	<u>06.2026</u> <i>Планируемая дата</i>		Нормоконтролер	
10.	Внешняя рецензия	<u>06.2026</u> <i>Планируемая дата</i>		Руководитель ВКР	Е.Ю. Оболенская
11.	Защита работы на ГЭК	<u>06.2026</u> <i>Планируемая дата</i>		Руководитель ВКР	Е.Ю. Оболенская

Руководитель квалификационной работы _____
(подпись, дата)

Е.Ю. Оболенская
(И.О. Фамилия)

Студент _____

М.Е. Дорошин

АННОТАЦИЯ

Расчетно-пояснительная записка посвящена процессу разработки программно-аппаратной системы автопилотирования автомобилей в городской среде. В исследовательской части рассмотрены различные особенности систем автопилотирования различных компаний, их особенности эксплуатации и применимость в разных средах. В конструкторской части определены аппаратная и программная часть макета автомобиля, алгоритмы локализации и навигации, веб интерфейс для управления. Технологическая часть посвящена разработке методики тестирования системы в разных условиях и системе удаленной прошивки и отладки программы микроконтроллера.

ABSTRACT

The settlement explanatory note is devoted to the development process of a hardware-software system for car autopiloting in a city environment. The research section examines various features of autopiloting systems from different companies, their operational characteristics, and applicability in different environments. The design section defines the hardware and software components of the car prototype, localization and navigation algorithms, and a web interface for control. The technological section focuses on developing a methodology for testing the system under various conditions, as well as a system for remote firmware updates and debugging of the microcontroller program.

РЕФЕРАТ

Расчетно-пояснительная записка состоит из 94 страниц, включающих в себя 12 рисунков, 2 таблиц, 15 источников и 4 приложения.

АВТОПИЛОТИРОВАНИЕ, АВТОНОМНОЕ ТРАНСПОРТНОЕ СРЕДСТВО, ГОРОДСКАЯ СРЕДА, МАКЕТ АВТОМОБИЛЯ, МИКРОКОНТРОЛЛЕР, КОМПЬЮТЕРНОЕ ЗРЕНИЕ, ЛИДАР, UWB-ПОЗИЦИОНИРОВАНИЕ, SLAM, V2X, HD-КАРТА, ПОСТРОЕНИЕ МАРШРУТА

Объектом разработки является программно-аппаратная система автопилотирования автомобилей в городской среде, реализованная на масштабном макете.

Цель работы — проектирование и реализация системы автопилотирования, включающей аппаратную часть макета автомобиля, алгоритмы локализации и навигации, подсистему распознавания объектов и элементов городской инфраструктуры, а также веб-интерфейс управления и систему удалённой отладки.

В процессе работы проведён аналитический обзор существующих систем автопилотирования компаний Waymo, Tesla, Cruise, Baidu Apollo и Yandex, выполнен их сравнительный анализ по ключевым критериям: типу сенсорной системы, использованию HD-карт, уровню автономности SAE, надёжности в городской среде, устойчивости к неблагоприятным погодным условиям, масштабируемости, стоимости аппаратной части, подходу к обучению моделей и степени интеграции с городской инфраструктурой.

Так как система разрабатывается как макет в закрытом помещении, использование реального GPS невозможно, поэтому вместо него используется UWB-радиомодуль в макете автомобиля и UWB-метки по углам макета города. UWB-модуль измеряет расстояние до меток по времени прохождения сигнала и определяет своё положение с помощью трилатерации.

Разработана аппаратная часть макета автомобиля на основе микроконтроллера STM32F411CEU6 и одноплатного компьютера Raspberry Pi Zero 2W. Аппаратная часть включает четыре тяговых мотора постоянного тока

с энкодерами, четыре сервопривода для рулевого управления, лидар LD06, UWB-модуль DWM1000, IMU ICM-54686 и камеру OV5647. Разработаны функциональная и принципиальная электрическая схемы устройства, произведена оценка потребляемой мощности.

Спроектирована многоуровневая программная архитектура с разделением вычислений между бортовым компьютером и удалённым сервером. На сервере реализованы подсистема распознавания объектов на основе модели YOLOv8n и подсистема построения маршрута с использованием HD-карты и алгоритма A*. На борту реализованы подсистема локализации, объединяющая UWB-позиционирование, лидарный SLAM и одометрию, и подсистема формирования управляющих команд по методу Pure Pursuit. Реализована демонстрация принципа V2X-коммуникации через двойное определение состояния светофора: по видеопотоку и через HTTP-запрос к контроллеру светофора.

Разработана система удалённой прошивки и отладки микроконтроллера через Raspberry Pi с эмуляцией протокола SWD через GPIO. Разработана методика тестирования системы в условиях деградации отдельных источников данных. Проведено комплексное тестирование в макете города, подтвердившее работоспособность системы в штатном режиме и её устойчивость к отказу любого одного источника данных.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	9
ВВЕДЕНИЕ	11
1 Анализ существующих систем автопилотирования	13
1.1 Аналитический обзор существующих систем автопилотирования	13
1.1.1 Разработки Waymo	13
1.1.2 Разработки Tesla	15
1.1.3 Разработки Cruise	17
1.1.4 Платформа Baidu Apollo	19
1.1.5 Разработки Yandex	20
1.2 Сравнительный анализ систем автопилотирования	23
1.2.1 Тип сенсорной системы	23
1.2.2 Использование HD-карт	24
1.2.3 Уровень автономности (SAE)	24
1.2.4 Надёжность в городской среде	25
1.2.5 Устойчивость к плохой погоде	25
1.2.6 Масштабируемость	25
1.2.7 Стоимость аппаратной части	26
1.2.8 Подход к обучению моделей	26
1.2.9 Интеграция с городской инфраструктурой	27
1.2.10 Обобщенные результаты	27
1.2.11 Выводы	29
2 Проектирование, разработка и тестирование системы	30
2.1 Описание функциональной схемы макета автомобиля	30
2.1.1 Обоснование выбора электронных компонентов	32
2.1.2 Описание архитектуры и характеристики STM32F411CEU6	35
2.1.3 Организация памяти микроконтроллера	36
2.1.4 Тактовая система микроконтроллера	36
2.1.5 Состав и основные характеристики периферийных модулей	36
2.1.6 Корпус и эксплуатационные условия микроконтроллера	37

2.2 Описание принципиальной электрической схемы	37
2.2.1 Подключение микроконтроллера к компьютеру	39
2.2.2 Подключение сервомо приводов	39
2.2.3 Подключение моторов	39
2.2.4 Подключение энкодеров	40
2.3 Оценка потребляемой мощности макета	41
2.4 Проектирование системы автопилотирования	42
2.4.1 Аппаратные компоненты системы автопилотирования	44
2.4.2 Программные компоненты системы автопилотирования	45
2.5 Разработка системы автопилотирования	46
2.5.1 Разработка подсистемы распознавания объектов	46
2.5.2 Разработка подсистемы построения маршрута	48
2.5.3 Разработка подсистемы управления макета автомобиля	49
2.6 Разработка Веб-интерфейса управления системой	51
2.6.1 Отображаемые данные	52
2.6.2 Пользовательский ввод	52
3 Разработка технологий удалённой прошивки и тестирования системы . . .	53
3.1 Технология удаленного обновления прошивки микроконтроллера . . .	53
3.2 Технология тестирования работы системы в разных условиях	54
3.2.1 Комплексное тестирование системы автопилотирования	55
ЗАКЛЮЧЕНИЕ	59
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	62
ПРИЛОЖЕНИЕ А. Техническое задание	64
ПРИЛОЖЕНИЕ Б. Руководство программиста	73
ПРИЛОЖЕНИЕ В. Фрагменты исходного кода компонентов	81
ПРИЛОЖЕНИЕ Г. Копии листов графической части	88

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В данной расчётно-пояснительной записке используются следующие определения и сокращения.

SAE – Society of Automotive Engineers, международная организация по стандартизации уровней автономности транспортных средств

HD-карта – высокодетализированная карта (High-Definition Map) для автономных транспортных средств, включающая точную информацию о дорожной разметке, светофорах, препятствиях и инфраструктуре

Lidar (Лидар) – Light Detection and Ranging, сенсор для измерения расстояний до объектов с помощью лазерного излучения

Radar (Радар) – радиоизмерительный сенсор, определяющий скорость и расстояние до объектов с помощью радиоволн

Vision-only – подход к автопилотированию, основанный исключительно на обработке данных с камер без использования лидаров или радаров

FSD – Full Self-Driving, пакет автопилотирования Tesla, реализующий элементы автономного управления

V2X – Vehicle to Everything, коммуникация транспортного средства с инфраструктурой, другими транспортными средствами и элементами умного города для повышения безопасности и предсказуемости движения

Автопилотирование – способность транспортного средства самостоятельно управлять движением без участия водителя

Сенсорная избыточность – использование нескольких типов сенсоров (лидар, радар, камеры) для повышения точности и надёжности восприятия окружающей среды

Сценарное моделирование – метод прогнозирования поведения участников движения с помощью моделирования дорожных ситуаций

Масштабируемость – способность системы к расширению зоны эксплуатации без существенного снижения производительности и безопасности

Инфраструктурная интеграция – взаимодействие автопилота с городской цифровой инфраструктурой, включая светофоры, дорожные датчики и системы мониторинга

Sensor fusion – объединение данных с разных сенсоров для более точного и надёжного восприятия окружающей среды

МК – микроконтроллер, микросхема для программного управления электронными устройствами

SWD – Serial Wire Debug, протокол программирования и отладки микроконтроллеров Arm Cortex-M

GPIO – General Purpose Input Output, универсальный порт ввода-вывода

CSI – Camera Serial Interface, интерфейс для подключения модулей камер, использует дифференциальные сигналы для передачи данных на большой скорости

ПИ-регулятор – пропорционально-интегральный регулятор, метод управления системами по измеренным и заданным целевым значениям

ШИМ – широтно-импульсная модуляция, способ изменения среднего уровня напряжения в цифровых системах

UART – Universal Asynchronous Receiver/Transmitter, асинхронный интерфейс передачи данных в цифровых схемах

ОЗУ – оперативное запоминающее устройство, тип памяти для хранения временных данных, такие как массивы и переменные

Флеш-память – тип энергонезависимой памяти, используется для хранения программ

ВВЕДЕНИЕ

Развитие систем автономного управления транспортными средствами является одним из наиболее активно развивающихся направлений в области робототехники и встраиваемых систем. Автономные автомобили потенциально способны существенно повысить безопасность дорожного движения, снизить нагрузку на водителя и повысить эффективность использования городской транспортной инфраструктуры. Ведущие технологические компании - Waymo, Tesla, Cruise, Baidu и Yandex - активно инвестируют в разработку подобных систем, демонстрируя коммерческие результаты в формате роботакси и систем помощи водителю.

Несмотря на значительный прогресс, достигнутый в данной области, задача создания надёжного автопилота для городской среды по-прежнему остаётся технически сложной. Городская среда характеризуется высокой плотностью движения, непредсказуемым поведением участников, сложной топологией дорожной сети, наличием светофоров, дорожных знаков и пешеходных переходов, а также изменчивыми условиями освещённости и погоды. Решение этих задач требует интеграции множества технологий: компьютерного зрения, дальномерных сенсоров, алгоритмов локализации и построения маршрута, систем управления в реальном времени и коммуникации с городской инфраструктурой.

Разработка полноценного автономного автомобиля сопряжена с высокими финансовыми и инженерными затратами, поэтому в исследовательских и образовательных целях широко применяются масштабные макеты, воспроизводящие ключевые аспекты городской среды. Такой подход позволяет отрабатывать алгоритмы и архитектурные решения в контролируемых условиях, сохраняя при этом реалистичность задачи.

Целью данной выпускной квалификационной работы является проектирование и реализация программно-аппаратной системы автопилотирования автомобилей в городской среде на основе масштабного макета. Система должна обеспечивать автономное движение макета автомобиля по заданному

маршруту в пределах макета города, обнаружение препятствий и реакцию на сигналы светофора.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести аналитический обзор существующих систем автопилотирования и сравнить их особенности;
- выбрать элементную базу и разработать аппаратную часть макета автомобиля, включая схемы управления моторами, сервоприводами и подключения сенсоров;
- спроектировать программную архитектуру системы с разделением задач между бортовым компьютером и сервером;
- реализовать подсистему распознавания объектов на основе данных камеры и лидара;
- реализовать подсистему локализации с объединением данных UWB-позиционирования и лидарного SLAM;
- реализовать подсистему построения маршрута с использованием HD-карты макета города;
- реализовать демонстрацию принципа V2X-коммуникации на примере взаимодействия со светофором;
- разработать систему удалённой прошивки и отладки микроконтроллера;
- разработать методику тестирования системы в условиях деградации отдельных источников данных из-за погодных условий;
- провести комплексное тестирование в макете города.

1 Анализ существующих систем автопилотирования

1.1 Аналитический обзор существующих систем автопилотирования

Современные системы автопилотирования строятся по многоуровневой архитектуре, включающей восприятие окружающей среды, локализацию, прогнозирование поведения объектов, планирование траектории и управление исполнительными механизмами автомобиля. Несмотря на общую логическую структуру, конкретные технические реализации существенно различаются.

1.1.1 Разработки Waymo

Компания Waymo реализует архитектуру автономного вождения, построенную на принципе максимальной сенсорной избыточности и строгой формализации операционной области. В основе системы лежит многослойная модель обработки данных, первый уровень которой заключается в получении данных с сенсоров, а последний - установка заданной скорости колёс и направления поворота. Каждый уровень функционирует с резервированием и взаимной проверкой результатов. Автомобили оснащаются лидарами различной дальности действия, радарами и комплексом камер кругового обзора. Такая конфигурация позволяет формировать высокоточную трёхмерную модель окружающего пространства, устойчивую к частичным отказам датчиков и временным помехам [1].

Ключевым элементом платформы является использование HD-карт, которые содержат детализированную геометрию дорожной инфраструктуры, параметры полос движения, положение бордюров, знаков, светофоров и других стационарных объектов. Процесс локализации основан на сопоставлении текущих сенсорных данных с картографической моделью, что обеспечивает сантиметровую точность позиционирования. В городской среде, где перекрёстки могут иметь сложную конфигурацию, а дорожная разметка частично скрыта, подобная точность критически важна для безопасного движения. Фотография автономного такси Waymo показана на рисунке 1.



Рисунок 1 – Автономное такси Waymo

Алгоритмы восприятия среды используют глубокие нейронные сети, обученные на больших массивах данных городского движения. Система выполняет детекцию, классификацию и отслеживание объектов, а также их семантическую сегментацию. Особое внимание уделяется прогнозированию поведения пешеходов и транспортных средств, включая оценку вероятности резких манёвров или неожиданного выхода человека на проезжую часть. Модели прогнозирования формируют несколько сценариев развития ситуации, что позволяет учитывать неопределённость.

Планирование движения строится на вероятностных алгоритмах, учитывающих динамические ограничения автомобиля, правила дорожного движения и прогноз поведения окружающих участников. Система выбирает траекторию, минимизирующую риск столкновения и обеспечивающую плавность движения. При этом стратегия часто ориентирована на консервативное поведение, что повышает безопасность, но может снижать среднюю скорость движения в плотном трафике.

Основным преимуществом подхода Waymo является высокая надёжность и точность работы в пределах заранее подготовленных зон эксплуатации. Однако зависимость от детального картографирования требует значительных временных и финансовых затрат при выходе на новые рынки. Таким образом,

модель компании можно охарактеризовать как высокотехнологичную, но инфраструктурно зависимую.

Система Waymo применяется преимущественно в формате роботакси. Компания развернула коммерческие сервисы автономных перевозок в отдельных городах США (например, в Финиксе и Сан-Франциско). Эксплуатация осуществляется в пределах строго определённых геозон с предварительно подготовленными HD-картами.

Автомобили работают на уровне автономности SAE 4, то есть способны передвигаться без участия водителя в пределах операционной зоны. Основная область применения – пассажирские перевозки в городской среде.

Особенности использования:

- заранее картографированные районы;
- дистанционный мониторинг оператором;
- постепенное расширение зоны эксплуатации;
- высокая степень регуляторного контроля.

Waymo ориентируется на модель полностью автономного такси без водителя в салоне.

1.1.2 Разработки Tesla

Компания Tesla применяет альтернативную концепцию автономного вождения, основанную преимущественно на визуальном восприятии окружающей среды. В системе Tesla Full Self-Driving используется многокамерная конфигурация, формирующая непрерывный видеопоток с круговым обзором [2]. В отличие от ряда конкурентов, компания отказалась от использования лидаров, аргументируя это стремлением приблизить алгоритмическое восприятие к человеческому способу управления автомобилем.

Обработка данных осуществляется масштабными нейросетевыми архитектурами, которые формируют внутреннее представление сцены в формате Bird's Eye View. Такой подход позволяет объединять данные с нескольких камер в единую координатную модель и упрощает последующее планирование траектории. Глубинные модели выполняют детекцию объектов,

сегментацию дорожного пространства, оценку глубины и отслеживание динамики движения других участников.

Существенным преимуществом является доступ к огромному массиву данных, получаемых с автомобилей пользователей по всему миру. Это обеспечивает постоянное обновление и обучение алгоритмов, позволяя учитывать разнообразные дорожные сценарии и редкие ситуации. Масштабное машинное обучение становится ключевым инструментом повышения качества системы без существенного изменения аппаратной части.

Локализация в меньшей степени зависит от HD-карт и опирается на анализ текущей сцены. Это повышает универсальность и потенциальную масштабируемость решения, поскольку не требуется детальное картографирование каждой новой территории. Однако такая стратегия предъявляет крайне высокие требования к точности алгоритмов компьютерного зрения, особенно в условиях плохой видимости.

Система автопилота Tesla используется массово на серийных автомобилях компании по всему миру. Она доступна владельцам автомобилей в виде пакетов Autopilot и Full Self-Driving (FSD). Фотография автомобиля Tesla Model 3 показана на рисунке 2.



Рисунок 2 – Автомобиль Tesla Model 3

Главной сильной стороной Tesla является масштабируемость и относительная экономичность аппаратной конфигурации. В то же время устойчивость к сложным погодным условиям и достижение полной автономности уровня 4 остаются техническими вызовами. Подход можно охарактеризовать как программно-ориентированный и эволюционный.

Применение охватывает:

- помощь водителю на автомагистралях;
- автоматическую парковку;
- движение в городских условиях с контролем со стороны водителя.

Несмотря на активное продвижение FSD, система официально относится к уровням SAE 2–3 и требует постоянного контроля водителя. Использование не ограничено географией и возможно в большинстве стран при соблюдении местных нормативов.

В отличие от большинства других систем, специализирующихся на такси и доставке, Tesla распространяет систему через потребительский рынок автомобилей с последующим обновлением программного обеспечения «по воздуху» (OTA).

1.1.3 Разработки Cruise

Компания Cruise реализует гибридную стратегию, сочетающую сенсорную избыточность и ограниченную операционную область. Архитектура системы включает лидары, радары и камеры, что обеспечивает комплексное восприятие окружающей среды. Сенсоры располагаются таким образом, чтобы зоны обзора перекрывались, обеспечивая устойчивость к частичным отказам и снижая вероятность «слепых зон» [3].

Система ориентирована на эксплуатацию в строго определённых городских районах, где инфраструктура тщательно изучена и протестирована. Это позволяет снизить уровень неопределённости и сосредоточить усилия на повышении безопасности. Ограничение географической зоны рассматривается не как недостаток, а как инструмент постепенного внедрения технологии.

Алгоритмы восприятия используют методы глубокого обучения для детекции и отслеживания объектов. Особое внимание уделяется анализу поведения пешеходов, поскольку именно они создают наибольшую неопределённость в городской среде. Модели прогнозирования учитывают плотность движения, расположение остановок общественного транспорта и другие контекстные факторы.

Планирование движения ориентировано на консервативную стратегию, минимизирующую риск конфликтов. Система предпочитает уступать дорогу и избегать агрессивных манёвров, даже если это снижает эффективность маршрута. Такой подход повышает общественное доверие и безопасность на этапе внедрения.

Cruise развивает автономные такси в ограниченных районах крупных городов США. Система используется в формате коммерческих роботакси, аналогично модели Waymo. Фотография автономного такси Cruise показана на рисунке 3.



Рисунок 3 – Автономное такси Cruise

Главным преимуществом Cruise является высокая степень отказоустойчивости и управляемость процесса внедрения. Ограничением остаётся

масштабируемость и зависимость от тщательно подготовленных зон эксплуатации.

Применение:

- перевозка пассажиров без водителя в пределах городской геозоны;
- работа преимущественно в ночное время на ранних этапах внедрения;
- строгий контроль условий эксплуатации.

Cruise делает акцент на постепенном расширении зон обслуживания и поэтапном тестировании сценариев городской среды.

1.1.4 Платформа Baidu Apollo

Платформа Apollo компании Baidu ориентирована на создание открытой и модульной экосистемы автономного транспорта. В отличие от полностью закрытых решений, Apollo предоставляет инструменты и программные модули для интеграции различными производителями [4]. Это способствует формированию широкой технологической базы и стандартизации компонентов.

Сенсорная архитектура включает лидары, камеры и радары, объединённые посредством алгоритмов *sensor fusion*. Большое внимание уделяется согласованию данных различных типов и построению единой модели окружающей среды. Такой подход обеспечивает баланс между точностью геометрического восприятия и семантической интерпретацией сцены.

HD-карты используются для повышения точности локализации, особенно в сложных городских условиях. Однако платформа также развивает методы локализации, способные функционировать при частичном отсутствии картографических данных, что повышает гибкость системы [5].

Особенностью Apollo является активное развитие V2X-коммуникаций. Автомобили могут получать данные от светофоров, дорожных датчиков и других элементов инфраструктуры. Это снижает неопределённость и повышает эффективность движения, особенно на регулируемых перекрёстках.

Платформа ориентирована на интеграцию в концепцию «умного города», где транспортные средства взаимодействуют с цифровой инфраструктурой.

Такой подход расширяет функциональные возможности системы, но требует развитой городской сети связи.

Особенности применения:

- интеграция с городской инфраструктурой;
- использование V2X-коммуникаций;
- тестирование в специально оборудованных районах.

Платформа Apollo компании Baidu применяется в Китае в рамках развития концепции «умного города». Основная сфера использования – автономные такси и экспериментальные транспортные сервисы в пилотных зонах. Фотография такси Baidu Apollo показана на рисунке 4.



Рисунок 4 – Автономное такси Baidu Apollo

Apollo также функционирует как технологическая платформа, предоставляемая партнёрам для разработки автономных транспортных решений.

1.1.5 Разработки Yandex

Компания Yandex разрабатывает автономные автомобили с учётом сложных климатических и инфраструктурных условий. Архитектура системы основана на мультисенсорной конфигурации, включающей камеры, лидары и радары. Такой подход обеспечивает устойчивость к разнообразным погодным факторам, включая снегопады и ограниченную видимость [6].

Алгоритмы адаптированы к зимним условиям эксплуатации. Используются методы фильтрации шумов от снегопада, корректировка моделей распознавания при частичном отсутствии разметки и механизмы очистки сенсоров. Это позволяет системе функционировать в условиях, которые часто оказываются критичными для визуально-ориентированных решений.

HD-карты активно используются для точной локализации в условиях плотной городской застройки. Сопоставление сенсорных данных с картографической моделью позволяет поддерживать высокую точность позиционирования даже при нестабильном спутниковом сигнале [7]. Фрагмент HD-карт Yandex показан на рисунке 5.



Рисунок 5 – HD-карты Yandex

Прогнозирование поведения участников движения строится на нейросетевых моделях, обученных на данных российских городов. Учитываются особенности локального стиля вождения и высокая вариативность дорожных сценариев [8].

Подход ориентирован на практическую эксплуатацию в реальных условиях, а не только в тщательно подготовленных зонах. Это повышает устойчивость к неопределённости, но требует значительных усилий по тестированию и адаптации алгоритмов.

Компания разрабатывает автономные наземные роботы, предназначенные для доставки еды, продуктов и посылок на «последней миле». Робот показан на рисунке 6.



Рисунок 6 – Робот доставки Yandex

Особенности применения:

- передвижение по тротуарам и пешеходным зонам;
- использование камер, лидаров и ультразвуковых датчиков;
- автономная навигация в городской среде с обходом препятствий;
- дистанционный контроль при возникновении нестандартных ситуаций.

Также система автономного управления Yandex применяется в формате роботакси в испытательных проектах в отдельных городах.

Особенности использования:

- автономные такси в пределах тестовых зон;
- испытания в сложных климатических условиях (зима, снег);
- адаптация к плотной городской застройке.

Yandex уделяет внимание устойчивости к неблагоприятным погодным условиям и вариативности дорожной среды.

1.2 Сравнительный анализ систем автопилотирования

Сравнительный анализ существующих систем автопилотирования в городской среде направлен на выявление ключевых различий в архитектурных решениях, алгоритмических подходах и стратегиях внедрения. Несмотря на общую цель, компании Waymo, Tesla, Cruise, Baidu и Yandex реализуют различные технологические концепции. Эти различия затрагивают как аппаратную конфигурацию транспортных средств, так и принципы обработки данных, обучения моделей и взаимодействия с городской инфраструктурой.

Для корректного сопоставления систем необходимо учитывать специфику городской среды, характеризующейся высокой плотностью движения, сложной топологией дорожной сети и значительным числом непредсказуемых факторов. В таких условиях особенно важными становятся надёжность сенсорной системы, точность локализации, устойчивость к погодным воздействиям, способность прогнозировать поведение участников движения, а также масштабируемость решения при расширении зоны эксплуатации. Кроме того, существенную роль играют экономические параметры, включая стоимость аппаратной части и требования к вычислительным ресурсам.

Сравнение проводится по ряду критериев, отражающих как технические характеристики систем, так и стратегические особенности их внедрения. Анализ позволяет выявить сильные и слабые стороны каждого подхода, определить уровень технологической зрелости решений и оценить перспективы их дальнейшего развития.

1.2.1 Тип сенсорной системы

Waymo, Cruise, Baidu Apollo и Yandex используют многосенсорную архитектуру, включающую лидары, радары и камеры. Такой подход обеспечивает сенсорную избыточность, что повышает надёжность распознавания объектов и устойчивость к частичным отказам отдельных сенсоров. Лидар обеспечивает точное измерение расстояний и построение трёхмерной модели сцены, радар – устойчивость к неблагоприятным погодным условиям, камеры – семантическое понимание окружающей среды.

Tesla реализует концепцию vision-only, опираясь преимущественно на камеры и алгоритмы компьютерного зрения. Отказ от лидаров и в значительной степени от радаров снижает стоимость аппаратной части и упрощает архитектуру, однако переносит основную нагрузку на алгоритмы обработки изображений и нейросетевые модели.

1.2.2 Использование HD-карт

Waymo активно использует высокодетализированные HD-карты, содержащие точную информацию о разметке, дорожной геометрии и инфраструктуре. Это обеспечивает высокую точность локализации, но требует предварительного картографирования зон эксплуатации.

Cruise и Yandex также применяют HD-карты в пределах операционных зон, ограничивая масштабирование необходимостью обновления картографических данных.

Baidu Apollo интегрирует HD-карты с городской инфраструктурой и системами «умного города», что усиливает предсказуемость среды.

Tesla минимизирует использование HD-карт, делая ставку на онлайн-восприятие окружающей среды в реальном времени. Такой подход повышает масштабируемость, но усложняет задачу обеспечения высокой точности локализации.

1.2.3 Уровень автономности (SAE)

Waymo, Cruise, Baidu и Yandex демонстрируют автономность уровня SAE 4 в ограниченных зонах эксплуатации (геозонах). Это означает способность автомобиля двигаться без участия водителя в заранее определённых условиях.

Tesla официально остаётся на уровне SAE 2, частично приближаясь к уровню 3 в рамках пакета FSD. Ответственность за управление по-прежнему лежит на водителе, что отличает подход Tesla от полностью автономных решений в геозонах.

1.2.4 Надёжность в городской среде

Cruise обеспечивает стабильную работу в пределах ограниченных городских зон.

Waymo демонстрирует очень высокую надёжность в тщательно подготовленных районах благодаря детализированным картам и сенсорной избыточности.

Baidu показывает высокую эффективность при наличии развитой инфраструктурной поддержки.

Yandex ориентируется на эксплуатацию в сложной и вариативной городской среде, включая плотную застройку.

Tesla демонстрирует переменную надёжность, существенно зависящую от условий видимости и корректности работы алгоритмов компьютерного зрения.

1.2.5 Устойчивость к плохой погоде

Многосенсорные системы Waymo, Cruise, Baidu и Yandex обладают высокой устойчивостью к дождю, туману и другим неблагоприятным условиям благодаря радарным и лидарным данным.

Yandex дополнительно адаптирует алгоритмы к зимним условиям, включая снег и обледенение, что является важным фактором для северных регионов.

Tesla, опирающаяся на камеры, в большей степени чувствительна к осадкам, засветке и снижению контрастности изображения.

1.2.6 Масштабируемость

Tesla обладает наибольшей масштабируемостью благодаря глобальному сбору данных с пользовательских автомобилей и отсутствию зависимости от предварительного картографирования.

Waymo и Cruise ограничены необходимостью детального картографирования каждой новой зоны.

Baidu и Yandex демонстрируют средний уровень масштабируемости, поскольку зависят от инфраструктурных решений и региональной адаптации.

1.2.7 Стоимость аппаратной части

Наиболее дорогостоящими являются решения с лидарной архитектурой (Waymo, Cruise). Использование нескольких сенсоров и высокопроизводительных вычислительных систем увеличивает себестоимость. Промышленные многолучевые лидары, применяемые в подобных системах, являются наиболее дорогостоящим компонентом сенсорной конфигурации.

Tesla имеет более низкую стоимость аппаратной части благодаря отказу от лидаров. Конфигурация из нескольких камер и ограниченного числа радаров значительно дешевле в производстве, что позволяет включать систему автопилотирования в комплектацию серийных потребительских автомобилей без существенного увеличения их цены. Экономия на аппаратной части при этом перекладывает нагрузку на программную составляющую, требуются более высокое качество обучающих данных и размер нейросетевых моделей.

Baidu и Yandex занимают промежуточное положение с умеренно высокой стоимостью. Их системы используют лидары, однако ориентированы на менее дорогостоящие конфигурации по сравнению с Waymo и Cruise, что обусловлено в том числе фокусом на масштабировании парка автомобилей в рамках коммерческих сервисов.

1.2.8 Подход к обучению моделей

Waymo и Cruise используют закрытые корпоративные данные и собственные испытательные парки автомобилей.

Tesla применяет массовый сбор данных с клиентских автомобилей, что обеспечивает огромный объём обучающей выборки и ускоряет совершенствование нейросетей.

Baidu развивает полуоткрытую платформу Apollo, предоставляя инструменты партнёрам.

Yandex активно использует собственные данные и симуляционные среды для отработки сценариев.

1.2.9 Интеграция с городской инфраструктурой

Baidu лидирует по уровню интеграции с инфраструктурой, развивая V2X-коммуникации и взаимодействие со светофорами и дорожными системами.

Waymo и Cruise ограниченно взаимодействуют с инфраструктурой, делая основной упор на автономность самого автомобиля.

Tesla практически не опирается на инфраструктурную интеграцию.

Yandex реализует частичную интеграцию, но в меньшем масштабе по сравнению с Baidu.

1.2.10 Обобщенные результаты

Для более наглядного и структурированного сравнения систем была составлена таблица с обобщёнными характеристиками систем автопилотирования по каждому из критериев, разобранных в предыдущих пунктах. Обобщённые результаты представлены в таблице 1. Далее, на основе этого сравнения, были сформулированы выводы.

Таблица 1 – Основные характеристики систем автопилотирования

Критерий	Waymo	Tesla	Cruise	Baidu Apollo	Yandex
Тип сенсорной системы	Лидары + радары + камеры	Камеры	Лидары + радары + камеры	Лидары + радары + камеры	Лидары + радары + камеры
Использование HD-карт	Активное, детализированные карты	Минимальное, ставка на онлайн-восприятие	Активное в пределах операционной зоны	Активное, интеграция с инфраструктурой	Активное, адаптация к сложной застройке
Уровень автономности (SAE)	Уровень 4 (в ограниченных зонах)	Уровень 2–3 (с элементами FSD)	Уровень 4 (в ограниченных зонах)	Уровень 4 (в ограниченных зонах)	Уровень 4 (в ограниченных зонах)

Продолжение таблицы 1

Критерий	Waymo	Tesla	Cruise	Baidu Apollo	Yandex
Надёжность в городской среде	Очень высокая в подготовленных зонах	Зависит от условий видимости	Высокая в ограниченной зоне	Высокая при поддержке инфраструктуры	Высокая в сложных погодных условиях
Устойчивость к плохой погоде	Высокая	Средняя (визуальная зависимость)	Высокая	Высокая	Очень высокая (адаптация к снегу и зиме)
Масштабируемость	Ограничена необходимостью картографирования	Высокая, глобальный сбор данных	Ограничена необходимостью картографирования	Средняя, зависит от инфраструктуры	Средняя
Стоимость аппаратной части	Высокая	Ниже средней	Высокая	Средняя–высокая	Средняя–высокая
Подход к обучению моделей	Закрытая система, собственные данные	Массовый сбор данных с пользователей	Закрытая модель, корпоративные данные	Полуоткрытая платформа	Собственные данные и симуляция
Интеграция с городской инфраструктурой	Ограниченная	Практически отсутствует	Ограниченная	Развитая V2X-коммуникация	Ограниченная

Продолжение таблицы 1

Критерий	Waymo	Tesla	Cruise	Baidu Apollo	Yandex
Основное преимущество	Точность и безопасность	Масштабируемость и программная эволюция	Контролируемая безопасность	Интеграция в «умный город»	Адаптация к сложным условиям
Основное ограничение	Высокая стоимость и зависимость от карт	Зависимость от компьютерного зрения	Ограниченность зон	Зависимость от инфраструктуры	Необходимость сложной адаптации

1.2.11 Выводы

Системы с лидарной архитектурой демонстрируют более высокую устойчивость к неблагоприятным условиям и большую точность локализации, однако их стоимость и сложность внедрения выше. Vision-only подход компании Tesla обеспечивает высокую масштабируемость и снижение аппаратных затрат, но предъявляет повышенные требования к алгоритмам обработки изображений.

Модели Waymo и Cruise ориентированы на безопасность и контролируемое внедрение, тогда как Baidu делает акцент на интеграцию с цифровой инфраструктурой города. Yandex выделяется адаптацией к сложным климатическим условиям и вариативной городской среде.

2 Проектирование, разработка и тестирование системы

2.1 Описание функциональной схемы макета автомобиля

Разрабатываемый макет автомобиля должен обеспечивать автономное движение по заданному маршруту в пределах макета города, обнаружение препятствий и реакцию на сигналы светофора. Исходя из этих требований, проведём анализ необходимых аппаратных компонентов.

Для реализации движения необходимы тяговые электродвигатели и механизм рулевого управления. Управление направлением и скоростью двигателей требует наличия микроконтроллера, формирующего управляющие сигналы, и драйверов моторов, обеспечивающих необходимые напряжение и ток. Для рулевого управления наиболее подходящим решением являются сервоприводы, позволяющие точно задавать угол поворота колёс.

Для реализации замкнутого контура управления скоростью микроконтроллеру необходима обратная связь о текущей скорости и пройденном расстоянии, что обеспечивается энкодерами на валах тяговых моторов.

Для локализации в закрытом помещении, где использование GPS невозможно, а также в масштабе макета 1/10 точность GPS будет в 10 раз хуже в сравнении с реальным автомобилем, необходим альтернативный метод позиционирования. Таким методом является UWB-позиционирование по стационарным меткам. Поскольку трилатерация между UWB метками определяет только координаты, но не ориентацию автомобиля, необходим IMU-сенсор, предоставляющий данные для определения ориентации.

Для обнаружения препятствий и локализации методом SLAM необходим лидар, обеспечивающий сканирование окружающего пространства в горизонтальной плоскости.

По результатам анализа требований к устройству можно сформулировать ряд компонентов, необходимых для работы устройства:

- микроконтроллер;
- драйверы моторов;
- моторы;

- энкодеры;
- сервомоторы;
- IMU сенсор;
- UWB трансивер;
- лидар.

Функциональная схема представлена на рисунке 7.

На функциональной схеме микроконтроллер STM32F411CEU6 изображён в развернутом виде: показаны ядро ARM Cortex-M4, блок программной флеш-памяти, внутренняя SRAM, таймеры, интерфейсы UART, SPI, и другие, модуль тактирования и система прерываний.

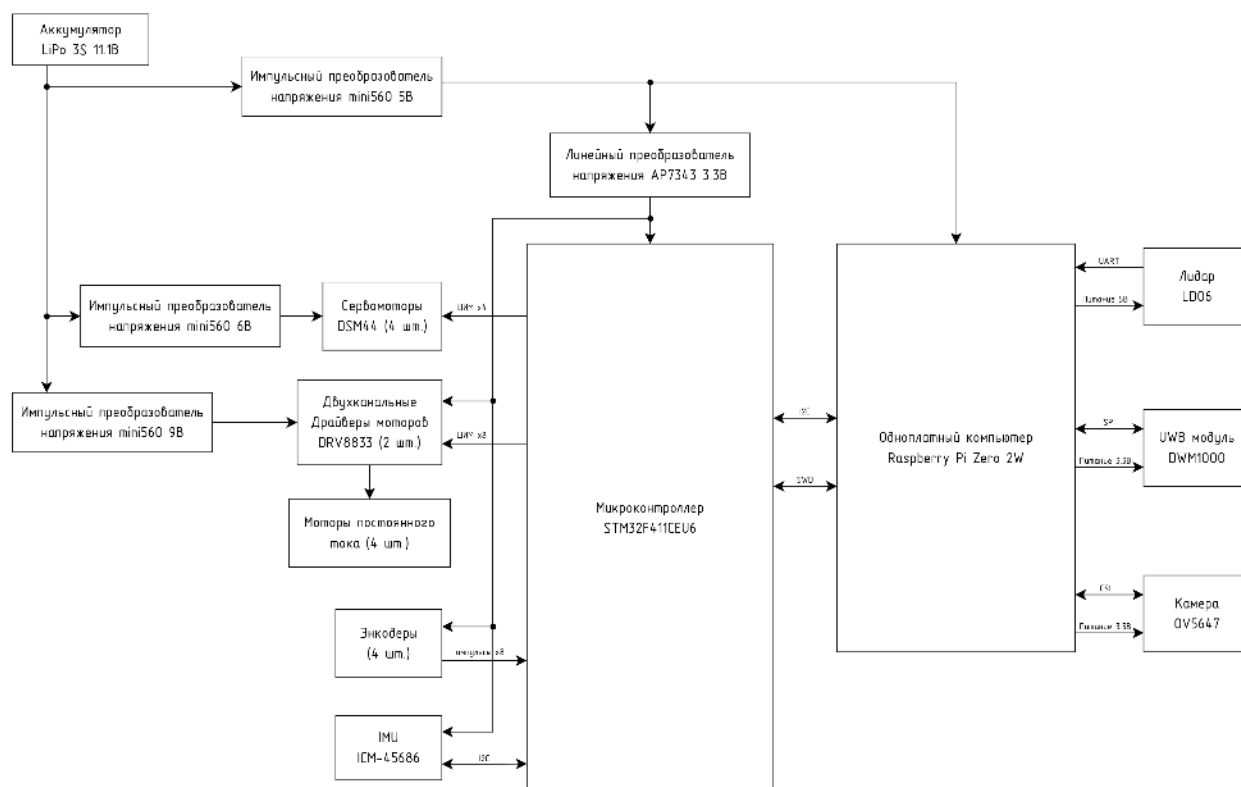


Рисунок 7 – Функциональная схема

На функциональной схеме представлена архитектура разрабатываемой системы управления машинкой. Система включает два основных вычислительных узла: высокоуровневый контроллер на базе одноплатного компьютера Raspberry Pi Zero 2W и низкоуровневый контроллер на базе микроконтроллера STM32F411CEU6.

Компьютер (Raspberry Pi) выполняет обработку видеопотока с камеры, реализацию алгоритмов компьютерного зрения, построение траектории

движения и генерацию команд управления. Связь между Raspberry Pi и STM32 осуществляется по последовательному асинхронному интерфейсу UART, обеспечивающему обмен управляющими пакетами и телеметрией.

Основная часть схемы – микроконтроллер STM32F411CEU6, изображенный с внутренними модулями.

Управление компонентами организовано через ШИМ, внешние прерывания, интерфейсы UART, SPI и I2C. ШИМ используются для регулировки скорости моторов. Прерывания нужны для обработки сигналов энкодеров, чтобы точно определять скорость и положение. Интерфейсы UART используются для связи с одноплатным компьютером и получения данных с лидара. SPI используется для коммуникации с UWB модулем, I2C используется для IMU сенсора.

Внутренняя организация МК показывает взаимодействие модулей через шинную матрицу АНВ. Ядро, флеш-память, SRAM и контроллеры DMA взаимодействуют по высокоскоростной шине. Периферийные модули (таймеры, UART, GPIO, SPI, I2C) подключены через мосты к шинам APB1 и APB2, что оптимизирует энергопотребление и разгружает основную шину.

Тактирование системы обеспечивается от внешнего кварцевого резонатора 8 МГц через внутренний PLL, что позволяет ядру работать на максимальной частоте 100 МГц. Цепь сброса и стабилизированное питание +3.3В обеспечивают стабильную работу всего устройства.

2.1.1 Обоснование выбора электронных компонентов

Raspberry PI Zero 2W был выбран, так как он имеет компактный размер 30×60мм, интерфейс CSI и Wi-Fi. Интерфейс CSI позволяет получать изображения с камеры с минимальной задержкой и не нагружает процессор, так как данные напрямую передаются в модуль обработки видео VideoCore IV внутри процессора BCM2710A1, в отличие от USB, где все данные сначала проходят через процессор. Была использована камера OV5647 [9] с углом обзора 120 градусов, разрешением 2592х1944 пикселей и частотой 90 кадров в секунду при разрешении 480р или 30 кадров в секунду при разрешении 1080р.

OV5647 официально поддерживается в Raspberry PI OS, что позволяет легко использовать все его режимы.

Привод рулевого управления состоит из четырех сервомоторов. Выбран именно такой тип моторов, так как он позволяет точно задавать угол поворота, что необходимо для выполнения маневров. В качестве сервомоторов выбраны Power HD DSM44. Это быстрые и точные сервомоторы компактного размера, они имеют крутящий момент 1.6 кг/см и скорость 850 °/с согласно документации [10], что позволяет точно маневрировать даже на большой скорости движения. В сервомоторах стоят встроенные драйверы моторов, поэтому ими можно управлять микроконтроллером напрямую. Сервомоторы подключены к линиям GPIO, работающим в режиме программной генерации ШИМ-сигнала с частотой 50 Гц, так как аппаратные выводы ШИМ были заняты другими компонентами. Управляющие импульсы формируются по прерываниям от таймера.

В качестве тяговых двигателей используются четыре мотора, подключенные к драйверам DRV8833. Моторы были выбраны постоянного тока стандартного форм-фактора N20 с редуктором и энкодером. Данный тип моторов может развивать достаточно быструю скорость и легко управляется с помощью ШИМ. Максимальная скорость выбранных моторов 760 об/мин, при диаметре колёс 25мм, это позволяет развить скорость 1 м/с. Напряжения питания и максимальный потребляемый ток моторов 9В и 500мА соответственно. Так как пины микроконтроллера не могут выдавать такое напряжение и ток, используются драйверы. Были выбраны драйверы DRV8833, так как они позволяют управлять направлением вращения мотора, поддерживают ШИМ для управления скоростью моторов и рассчитаны на напряжение от 2.7 до 10.8 В и ток до 2А [11]. Управление осуществляется через аппаратные ШИМ-каналы таймеров TIM1 и TIM4.

Для обратной связи используются инкрементальные энкодеры, установленные на валах двигателей, подключенные к входам GPIO. Их сигналы обрабатываются с помощью прерываний, что позволяет точно подсчитывать

импульсы и определять текущую скорость и пройденный путь. Конкретно используются магнитные энкодеры на датчиках холла, поставляемые вместе с выбранными моторами постоянного тока.

В качестве модуля UWB-позиционирования выбран DWM1000 производства Qorvo. Модуль реализует двустороннее дальномерное измерение по времени прохождения сигнала (TWR), обеспечивая точность определения расстояния 2-5 см [12]. Один модуль DWM1000 установлен на макете автомобиля и подключён к микроконтроллеру STM32F411CEU6 по интерфейсу SPI. Четыре стационарные метки DWM1000 размещены по периметру макета города в точках с известными координатами и работают в режиме ответчика. По измеренным расстояниям до меток бортовой модуль вычисляет свои координаты методом трилатерации. Использование UWB обусловлено невозможностью применения GPS в закрытом помещении и в масштабах макета, UWB обеспечивает точность позиционирования в условиях макета сопоставимую с точностью GPS-позиционирования для реального автомобиля с хорошим сигналом GPS.

В качестве IMU выбран датчик ICM-45686 производства TDK InvenSense. Датчик содержит трёхосевой акселерометр и трёхосевой гироскоп в одном корпусе. Диапазон измерения угловой скорости составляет до 4000 °/с, диапазон измерения ускорения — до 32g [13], что с запасом перекрывает требования ТЗ (2000 °/с и 40 м/с²). Датчик подключен к микроконтроллеру STM32F411CEU6 по интерфейсу I2C. В системе IMU используется совместно с UWB-модулем: трилатерация по UWB меткам определяет координаты автомобиля, а ориентация определяется с помощью фильтра Маджвика по данным IMU сенсора.

В качестве лидара выбран LD06 производства LDRobot. Это компактный одноплоскостной лидар с дальностью сканирования до 12 метров, частотой вращения 10 Гц и угловым разрешением 1°, обеспечивающий до 4500 измерений в секунду [14]. Лидар подключен к микроконтроллеру STM32F411CEU6 по интерфейсу UART. Данные сканирования используются в двух подсистемах:

для обнаружения препятствий в подсистеме распознавания объектов и для локализации методом SLAM в подсистеме построения маршрута. Выбор LD06 обусловлен его компактными габаритами, совместимыми с масштабом макета, и достаточной дальностью для перекрытия всей площади макета города.

2.1.2 Описание архитектуры и характеристики STM32F411CEU6

Этот микроконтроллер имеет 512КБ флеш-памяти, 128КБ ОЗУ, 32-битное ядро Arm Cortex-M4 работающее на частоте 100МГц, FPU для работы с 32-битными числами с плавающей точкой, аппаратную поддержку ШИМ на 24 пинах одновременно с помощью таймеров, прерывания на любых вводах и интерфейс UART. Всё это необходимо для быстрых расчётов траектории, точного управления всеми моторами и обработки данных с энкодеров. Также он поддерживает интерфейсы SPI и I2C, они используются для подключения UWB-модуля и IMU соответственно.

Блок вычислений с плавающей запятой (FPU) единичной точности, поддерживает все инструкции 32-битной арифметики с плавающей точкой, что существенно ускоряет выполнение алгоритмов цифровой фильтрации, преобразований и других математических операций. Набора инструкций Thumb-2, используемый этим микроконтроллером, обеспечивает высокую плотность кода при сохранении производительности. Arm Cortex-M4 реализован на архитектуре Гарвардского типа с тремя отдельными шинами: для инструкций, данных и системной периферии, что позволяет выполнять выборку команд и доступ к данным одновременно.

Ядро работает на тактовой частоте до 100 МГц, обеспечивая производительность до 125 MIPS (Dhrystone Million Instructions Per Second)

Микроконтроллер использует шины для передачи данных между внутренними компонентами. 2 шины Advanced High-performance Bus (AHB) используются для обеспечения высокой пропускной способности между ядром, памятью и DMA, а 2 шины Advanced Peripheral Bus (APB) подключают периферию к шинам AHB через мосты.

2.1.3 Организация памяти микроконтроллера

Микроконтроллер имеет флеш-память для хранения программ и оперативную SRAM память.

флеш-память программы объемом 512 КБ. Организована для хранения исполняемого кода и константных данных. Имеет кэш инструкций, который, в сочетании с 128-битной шиной, обеспечивает выполнение большинства инструкций за один такт даже на максимальной частоте.

Оперативная память (SRAM) объемом 128 КБ. Используется для хранения переменных, стека и динамических данных. Часть SRAM может быть сконфигурирована для работы в режиме сохранения данных при переходе в режим низкого энергопотребления (режим сна с сохранением SRAM).

2.1.4 Тактовая система микроконтроллера

Для гибкости и энергоэффективности реализована продвинутая система тактирования:

Микроконтроллер имеет внутренние RC-генераторы высокой (16 МГц) и низкой (32 кГц) частоты, возможность использования внешних кварцевых резонаторов: высокочастотного (до 26 МГц) и низкочастотного (32768 Гц) для повышения точности таймеров и работы в реальном времени. Есть модуль фазовой автоподстройки частоты (PLL) для умножения частоты внутренних или внешних источников до требуемого значения.

2.1.5 Состав и основные характеристики периферийных модулей

STM32F411CEU6 обладает широким набором периферии, актуальной для задач микропроцессорных систем:

12-разрядный АЦП с временем преобразования 0.2 мкс, поддерживающий последовательное преобразование до 16 внешних каналов.

Таймеры:

- 2 32-разрядных таймера общего назначения;
- 5 16-разрядных таймеров общего назначения;
- 1 таймер с расширенным управлением (для сложных схем ШИМ);

- 1 системный таймер (SysTick);
- 2 сторожевых таймера (Watchdog).

Последовательные интерфейсы связи:

- 3 интерфейса USART/UART (с поддержкой LIN и IrDA);
- 5 интерфейсов SPI (с поддержкой протокола I2S);
- 3 интерфейса I2C (с поддержкой SMBus и PMBus).

Интерфейсы для подключения внешних устройств и памяти:

- SDIO/MMC интерфейс для работы с картами памяти;
- USB 2.0 Full-Speed с поддержкой режимов Host и Device.

До 81 порта ввода/вывода общего назначения (GPIO) с программируемыми настройками (подтяжка, скорость, режим), большинство из которых поддерживает работу с 5В в режиме ввода.

Интерфейсы Serial Wire Debug (SWD) и JTAG для программирования и отладки.

2.1.6 Корпус и эксплуатационные условия микроконтроллера

Микроконтроллер выпускается в разных корпусах, был выбран вариант в корпусе UFQFPN48 с размерами 7x7 мм. Это компактный корпус с 48 пинами, что ограничивает доступность всех периферийных функций одновременно, но делает устройство оптимальным для проектов с требованиями к габаритам. Такой вариант был выбран так как он более распространен и 48 пинов достаточно для выполнения задач этой дипломной работы. Рабочий диапазон напряжения питания: 1.7 В до 3.6 В. Диапазон рабочих температур: –40°C до +85°C.

2.2 Описание принципиальной электрической схемы

На принципиальной схеме центральным элементом является микроконтроллер STM32F411CEU6 (DD1), к выводам которого подведены питание, схемы тактирования и сброса, интерфейсы программирования и связи с компьютером, линии управления сервомоторами и драйверами моторов, энкодеры. Принципиальная схема устройства представлена на рисунке 8.

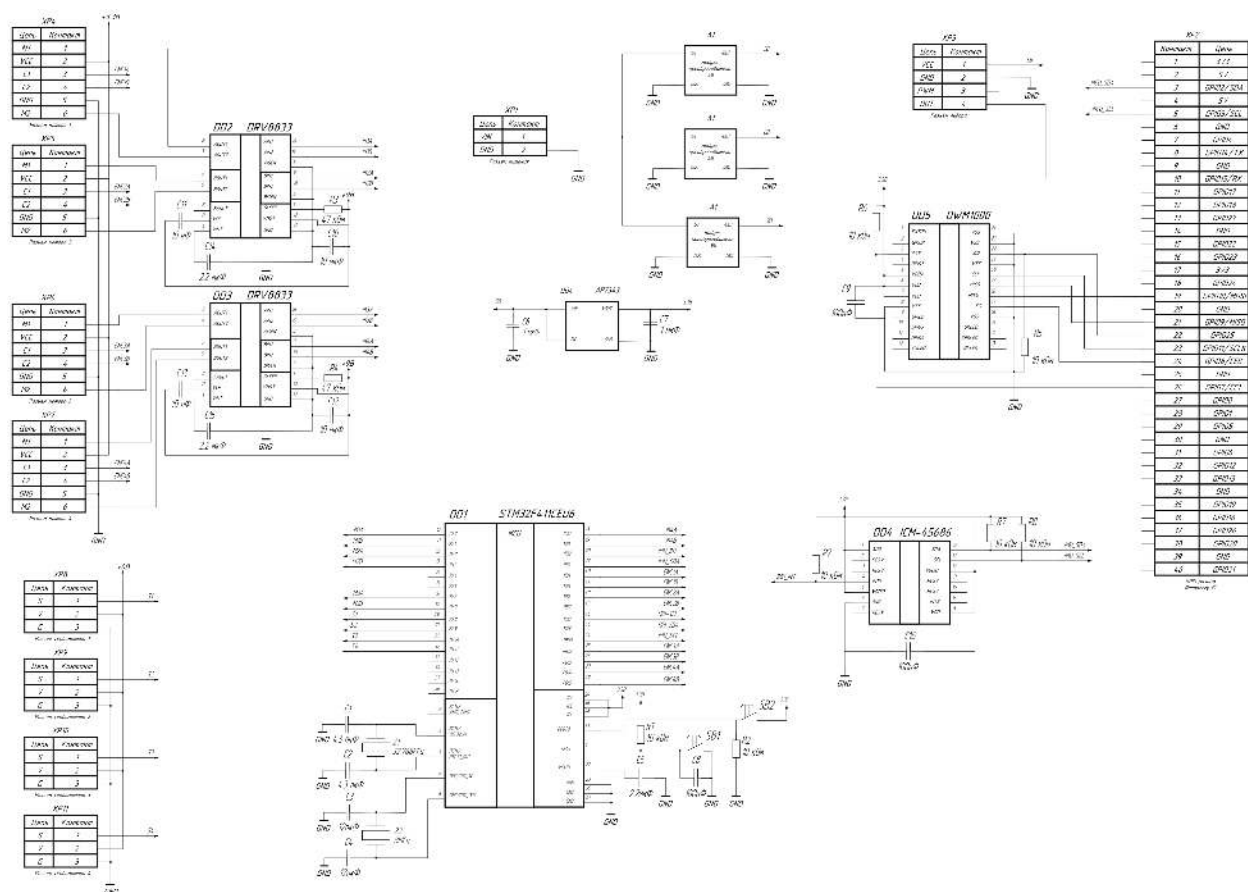


Рисунок 8 – Принципиальная схема

Питание устройства осуществляется от линий +5В, +6В и +9В, поступающих с преобразователей, получающих напряжение от аккумулятора. Напряжение +5В преобразуется в +3.3В с помощью линейного преобразователя. Вблизи выводов питания микроконтроллера и других микросхем размещены развязывающие керамические конденсаторы. Тактовый генератор построен на кварцевом резонаторе Z1 частотой 8 МГц, подключённом к выводам тактирования микроконтроллера. Также для отсчёта реального времени есть резонатор Z2 на 32.768 кГц. Конденсаторы небольшой ёмкости (4.3 и 12 пФ) согласно требованиям из документации к микроконтроллеру [15] (C1-C4) образуют с резонаторами пьезоэлектрический генератор.

Резисторы и конденсаторы подключенные к драйверам DRV8833, IMU сенсору ICM-45686 и UWB трансиверу DW1000 выбраны согласно документации на эти компоненты. Конденсаторы выполняют функции сглаживания высокочастотных импульсов, а резисторы используются для подтяжки сигналов для выбора режимов работы.

Цепь сброса включает подтягивающий резистор R1, соединяющий вывод RESET с линией +3.3 В, и кнопку SB1, замыкающую этот вывод на общую землю; при нажатии кнопки микроконтроллер переходит в состояние сброса. Также для сглаживания дребезга при нажатии на кнопку сброса стоит конденсатор C5. Для ручного перехода в режим прошивки есть кнопка SB2, она нужна для перехода в режим прошивки, если программно это сделать не удастся из-за ошибки программирования.

2.2.1 Подключение микроконтроллера к компьютеру

Для удобства разработки и отладки микроконтроллер поддерживает внутрисхемное программирование по SWD и JTAG. На схеме показан разъём программирования по интерфейсу SWD. Его выводы SWIO и SWCLK подключены к соответствующим выводам микроконтроллера.

Для передачи данных между компьютером и МК используется UART.

Одноплатный компьютер Raspberry PI Zero 2W, используемый в этом проекте, поддерживает UART аппаратно и может программно эмулировать SWD с помощью GPIO, что убирает необходимость использовать отдельный программатор или преобразователь USB-UART.

2.2.2 Подключение сервоприводов

Сервомоторы управляются с помощью широтно-импульсной модуляции с частотой 50 Гц. Угол поворота задается периодом ШИМ: 0 градусов - 500 мкс, 180 градусов - 2500 мкс. Центральное положение (90 градусов) при этом 1500 мкс.

Так как частота ШИМ не большая, можно использовать любые пины микроконтроллера, не обязательно с прямым выводом ШИМ от таймера, это позволяет более аккуратно развести макетную плату. ШИМ генерируется с помощью программного переключения GPIO портов по прерываниям таймера.

2.2.3 Подключение моторов

Для моторов постоянного тока необходимо использование драйверов, так как они работают от большего напряжения и тока чем может выдать микроконтроллер на портах ввода-вывода.

Драйверы моторов принимают сигналы от микроконтроллера, усиливают, и передают на моторы. Для управления направлением движением меняется полярность, $IN1=0$, $IN2=1$ - в одну сторону, $IN1=1$, $IN2=0$ - в другую. Для управления скоростью применяется ШИМ. По документации на драйвер, ШИМ можно использовать только на 1 порту, когда при этом на другой приходит логическая единица.

Для управления моторами используется аппаратный ШИМ микроконтроллера на таймерах 1 и 4.

2.2.4 Подключение энкодеров

Используются магнитные энкодеры на датчиках холла поставляемые в комплекте с выбранными моторами.

При вращении вала энкодер генерирует на своих выходах последовательность прямоугольных импульсов, смещённых относительно друг друга на $1/4$ периода (90 градусов). Это смещение создает фазовый сдвиг между сигналами каналов А и В, который является критерием определения направления вращения.

Выходы энкодеров подключены к пинам GPIO микроконтроллера и обрабатываются с помощью прерываний.

Собранный макет автомобиля показан на рисунке 9.

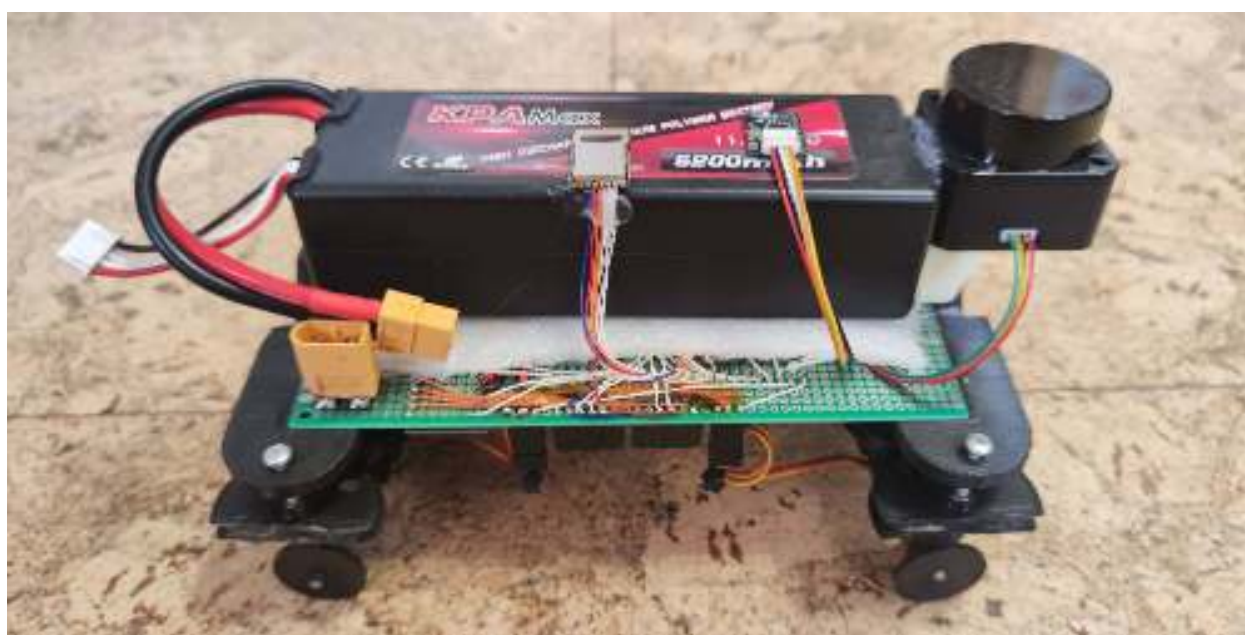


Рисунок 9 – Макет автомобиля

2.3 Оценка потребляемой мощности макета

Для оценки энергопотребления устройства требуется определить суммарный ток, потребляемый всеми узлами при работе в типовом режиме и при максимальной нагрузке. Учтём цифровые схемы, силовые устройства и потери в стабилизаторах напряжения.

Микроконтроллер STM32F411CEU6 (100 МГц с активной периферией):

Рассеиваемая мощность: $38 \text{ мА при } 3.3\text{В} \approx 125\text{мВт}$.

Драйверы DRV8833 (логическая часть, 4 шт.):

Рассеиваемая мощность: $2 \text{ мА при } 3.3\text{В} \approx 7\text{мВт}$.

Лидар LD06:

Рассеиваемая мощность: $480 \text{ мА при } 5\text{В} \approx 2400 \text{ мВт}$.

UWB-модуль DWM1000:

Рассеиваемая мощность: $120 \text{ мА при } 3.3\text{В} \approx 396 \text{ мВт}$.

IMU ICM-45686:

Рассеиваемая мощность: $3 \text{ мА при } 3.3\text{В} \approx 10 \text{ мВт}$.

Магнитные энкодеры (4 шт., датчики Холла):

Рассеиваемая мощность: $7 \text{ мА} \times 4 = 28 \text{ мА при } 3.3\text{В} \approx 92\text{мВт}$.

Raspberry Pi Zero 2W (с Wi-Fi и камерой):

Рассеиваемая мощность: $500 \text{ мА при } 5\text{В} \approx 2500\text{мВт}$.

Сервомоторы (4 шт.):

Мощность в покое: $10 \text{ мА} \times 4 = 40 \text{ мА при } 6\text{В} \approx 240\text{мВт}$.

Мощность под нагрузкой: $200 \text{ мА} \times 4 = 800 \text{ мА при } 6\text{В} \approx 4800\text{мВт}$.

Моторы постоянного тока (4 шт.):

Мощность холостого хода: $50 \text{ мА} \times 4 = 200 \text{ мА при } 9\text{В} \approx 1800\text{мВт}$.

Мощность под нагрузкой: $500 \text{ мА} \times 4 = 2000 \text{ мА при } 9\text{В} \approx 18000\text{мВт}$.

AP7343 (5В → 3.3В): Линейный стабилизатор.

Потребление на линии +3.3В: $38 \text{ мА} + 2 \text{ мА} + 28 \text{ мА} + 120 \text{ мА} + 3 \text{ мА} = 191 \text{ мА}$.

Рассеиваемая мощность: $(5 \text{ В} - 3.3 \text{ В}) \times 191 \text{ мА} \approx 325 \text{ мВт}$.

JW5069A (12В → 5В): Импульсный стабилизатор, КПД 95%.

Потребление на линии +5В: $2500\text{мВт} + 116\text{ мВт} = 2616\text{мВт}$.

Рассеиваемая мощность: $2616\text{мВт} \times (1/0.95-1) \approx 137\text{мВт}$.

JW5069A (12В → 6В): Импульсный стабилизатор, КПД 96%.

Потребление на линии +6В: 4800мВт .

Рассеиваемая мощность: $4800\text{мВт} \times (1/0.96-1) \approx 200\text{мВт}$.

JW5069A (12В → 9В): Импульсный стабилизатор, КПД 98%.

Потребление на линии +9В: 18000мВт .

Рассеиваемая мощность: $18000\text{мВт} \times (1/0.98-1) \approx 367\text{мВт}$.

Расчет суммарной мощности на входе от аккумулятора 12В:

Режим ожидания/покоя: (моторы постоянного тока выключены): $125\text{мВт} + 7\text{мВт} + 92\text{мВт} + 2500\text{мВт} + 240\text{мВт} + 116\text{ мВт} + 137\text{мВт} + 200\text{мВт} + 367\text{мВт} + 2400\text{мВт} + 396\text{мВт} + 10\text{мВт} \approx 6.8\text{ Вт}$.

Типовой режим движения с постоянной скоростью: $125\text{мВт} + 7\text{мВт} + 92\text{мВт} + 2500\text{мВт} + 240\text{мВт} + 1800\text{мВт} + 116\text{ мВт} + 137\text{мВт} + 200\text{мВт} + 367\text{мВт} + 2400\text{мВт} + 396\text{мВт} + 10\text{мВт} \approx 8.6\text{ Вт}$.

Пиковый режим: $125\text{мВт} + 7\text{мВт} + 92\text{мВт} + 2500\text{мВт} + 4800\text{мВт} + 18000\text{мВт} + 116\text{ мВт} + 137\text{мВт} + 200\text{мВт} + 367\text{мВт} + 2400\text{мВт} + 396\text{мВт} + 10\text{мВт} \approx 29.4\text{ Вт}$.

2.4 Проектирование системы автопилотирования

Поскольку вычислительные ресурсы одноплатного компьютера Raspberry Pi Zero 2W недостаточны для выполнения задач полноценного автопилотирования в реальном времени — в частности, для запуска нейросетевых моделей распознавания объектов и алгоритмов построения маршрута — была принята архитектура с выносом основных вычислений на отдельный сервер. Макет автомобиля связывается с сервером по протоколу Wi-Fi. Сервер находится в одной локальной сети с макетом. Такой подход позволяет использовать полноценные вычислительные мощности

без ограничений, накладываемых массой, габаритами и энергопотреблением бортового оборудования макета.

Взаимодействие между макетом и сервером организовано следующим образом. Raspberry Pi Zero 2W захватывает видеопоток с камеры по интерфейсу CSI, получает данные от лидара по UART и от UWB модуля по SPI, а также получает от микроконтроллера STM32F411CEU6 по I2C агрегированный пакет телеметрии, включающий данные энкодеров, и IMU. Все эти данные передаются на сервер по UDP. Сервер выполняет распознавание объектов, определение состояния светофора и построение маршрута, после чего возвращает на Raspberry Pi рассчитанную траекторию также по UDP. Raspberry Pi самостоятельно формирует управляющие команды для микроконтроллера на основе полученной траектории и текущих сенсорных данных и передаёт их микроконтроллеру по I2C.

Для всего обмена данными между бортом и сервером используется протокол UDP. Это обосновано тем, что траектория пересчитывается сервером после каждого полученного пакета телеметрии и кадра камеры, поэтому потеря нескольких пакетов с траекторией не является критичной: вместо них придёт следующий актуальный пакет. Накладные расходы протокола TCP и задержки, связанные с механизмом повторной передачи, в данном случае нежелательны.

Лидар LD06 и UWB-модуль DWM1000 подключены непосредственно к микроконтроллеру STM32F411CEU6, а не к Raspberry Pi. Такое решение обусловлено двумя причинами. Во-первых, Raspberry Pi Zero 2W располагает только одним аппаратным портом UART, который занят каналом обмена с микроконтроллером. Во-вторых, управление интерфейсом SPI из пространства пользователя Linux сопряжено со значительными задержками и джиттером, неприемлемыми для своевременного считывания данных UWB-модуля. Микроконтроллер, работающий в режиме реального времени, обеспечивает детерминированное считывание данных с обоих сенсоров и включает их в пакет телеметрии, отправляемый на Raspberry Pi по UART.

Система автопилотирования состоит из аппаратных и программных компонентов, распределённых между бортовой частью макета, макетом города и сервером.

2.4.1 Аппаратные компоненты системы автопилотирования

Камера OV5647 подключена к Raspberry Pi по интерфейсу CSI и обеспечивает захват видеопотока окружающей среды с углом обзора 120 градусов. Видеопоток используется для детекции дорожной разметки, объектов и состояния светофора на сервере.

Лидар LD06 подключён к микроконтроллеру STM32F411CEU6 по интерфейсу UART и обеспечивает сканирование окружающего пространства в горизонтальной плоскости с частотой 10 Гц и дальностью до 12 метров. Данные сканирования считываются микроконтроллером и включаются в пакет телеметрии. Помимо детекции препятствий, данные лидара используются для локализации автомобиля методом SLAM.

UWB-модуль DWM1000 подключён к микроконтроллеру STM32F411CEU6 по интерфейсу SPI. Модуль выполняет дальномерные измерения до стационарных меток DWM1000, размещённых по периметру макета города, и реализует функцию локальной системы позиционирования. Результаты измерений считываются микроконтроллером и включаются в пакет телеметрии наряду с данными энкодеров и лидара.

Одноплатный компьютер Raspberry Pi Zero 2W является центральным бортовым вычислителем. Он обеспечивает захват и передачу видеопотока на сервер, приём агрегированной телеметрии от микроконтроллера по UART, передачу телеметрии на сервер, приём траектории с сервера по UDP, формирование управляющих команд и их передачу микроконтроллеру по UART.

Микроконтроллер STM32F411CEU6 выполняет две группы задач. Первая группа — низкоуровневое управление приводами: регулирование скорости тяговых моторов посредством ПИ-регулятора и управление сервоприводами рулевого управления. Вторая группа — сбор данных с

периферийных сенсоров: считывание импульсов энкодеров, данных лидара LD06 по UART и дальномерных измерений UWB-модуля DWM1000 по SPI. Все собранные данные агрегируются в единый пакет телеметрии и передаются на Raspberry Pi по UART.

Светофор макета города оснащён контроллером, поддерживающим HTTP-интерфейс для получения его текущего состояния. Это позволяет реализовать демонстрацию принципа V2X-коммуникации: сервер может получать состояние светофора не только через анализ видеопотока с камеры автомобиля, но и напрямую по сети через HTTP-запрос к контроллеру светофора, что соответствует концепции взаимодействия транспортного средства с элементами городской инфраструктуры.

Стационарные UWB-метки DWM1000 размещены в фиксированных точках макета города с заранее известными координатами. Каждая метка работает в режиме ответчика и обеспечивает дальномерные измерения по запросу бортового модуля.

2.4.2 Программные компоненты системы автопилотирования

На сервере функционируют три программные подсистемы. Подсистема распознавания объектов и локализации принимает видеопоток, данные лидара и положение по UWB меткам с автомобиля, выполняет распознавание препятствий и дорожной разметки, определение состояния светофора и передачу результатов подсистеме построения маршрута. Подсистема взаимодействия с инфраструктурой получает состояние светофора через HTTP-запрос к его контроллеру и передаёт эти данные подсистеме построения маршрута в качестве дополнительного источника информации. Подсистема построения маршрута формирует траекторию движения на основе HD-карты макета, текущего положения автомобиля, обнаруженных объектов и состояния светофора, после чего передаёт траекторию на Raspberry Pi по UDP.

На борту автомобиля, на Raspberry Pi, функционируют две программные подсистемы. Подсистема сбора и передачи данных отвечает за захват видеопотока с камеры, приём телеметрического пакета от микроконтроллера

по UART и отправку всех данных на сервер по UDP. Подсистема формирования управляющих команд получает траекторию и положение автомобиля с сервера по UDP и формирует целевую скорость и угол поворота, которые передаются микроконтроллеру по UART.

На микроконтроллере STM32F411CEU6 функционируют две программные подсистемы. Подсистема управления приводами реализует ПИ-регуляторы скорости моторов и формирует ШИМ-сигналы для драйверов моторов и сервоприводов. Подсистема сбора данных датчиков считывает данные энкодеров, лидара, IMU и UWB-модуля, агрегирует их в пакет фиксированной структуры и отправляет на Raspberry Pi по UART с заданной периодичностью.

Функциональная схема программных компонентов системы автопилотирования показана на рисунке 10.

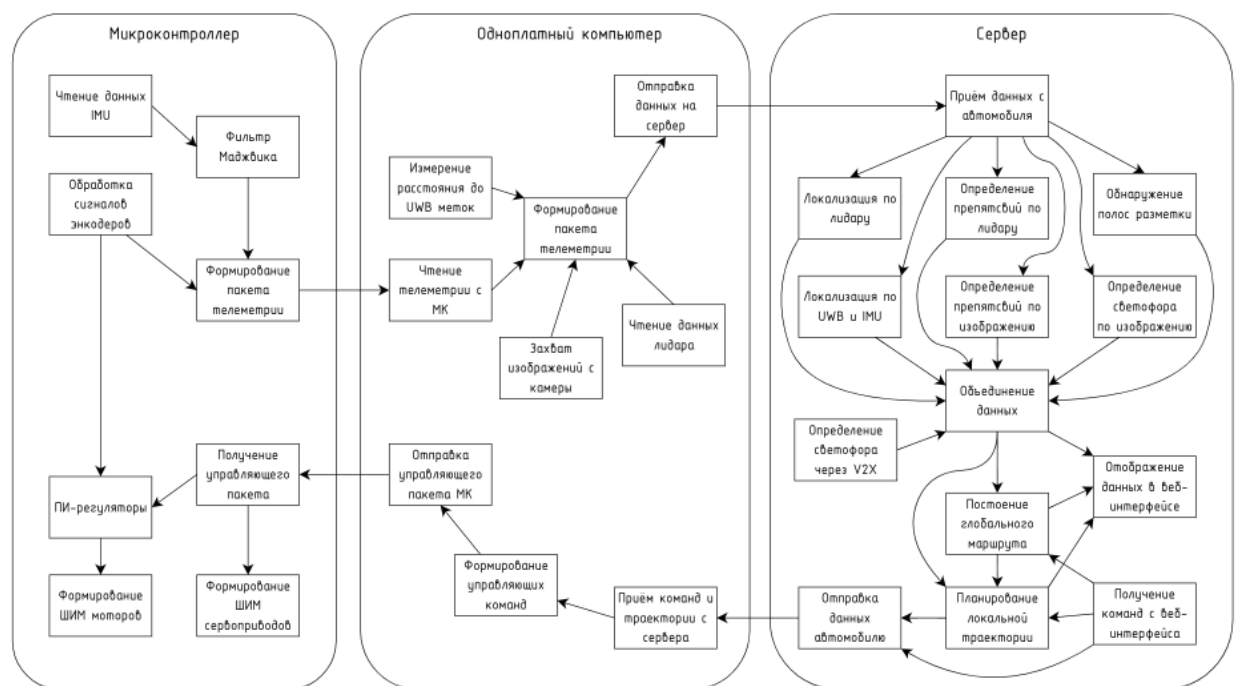


Рисунок 10 – Функциональная схема ПО

2.5 Разработка системы автопилотирования

2.5.1 Разработка подсистемы распознавания объектов

Подсистема распознавания объектов функционирует на сервере и получает на вход видеопоток с камеры и данные сканирования лидара LD06,

извлечённые из телеметрического пакета. Основными задачами подсистемы являются обнаружение полос дорожной разметки, детекция препятствий и определение состояния светофора.

Для детекции объектов применяется предобученная нейросетевая модель YOLOv8n. Модель обучена на датасете COCO и способна распознавать широкий набор объектов, встречающихся в городской среде, в том числе применительно к масштабам макета — пешеходов, транспортные средства и элементы дорожной инфраструктуры. В рамках данной работы дообучение модели на собственном датасете макета не проводилось. Вместе с тем дообучение YOLOv8n на изображениях, собранных непосредственно в условиях конкретного макета, может существенно повысить точность и уверенность детекции для характерных объектов данной среды и является перспективным направлением дальнейшего совершенствования системы.

Определение состояния светофора по видеопотоку выполняется отдельным алгоритмом. В кадре выделяется область интереса, соответствующая расположению светофора. В этой области выполняется анализ цветовых сегментов в пространстве HSV: красный и зелёный диапазоны пороговой бинаризацией, после чего по площади активных пикселей определяется текущий сигнал светофора.

Обнаружение полос разметки выполняется на основе классической обработки изображений. Входной кадр переводится в цветовое пространство HSV, после чего выполняется пороговая бинаризация для выделения белых полос. К бинаризованному изображению применяется детектор границ Canny и преобразование Хафа для нахождения прямых линий разметки. На основе найденных линий вычисляется отклонение автомобиля от центра полосы.

Данные лидара LD06 обрабатываются независимо от видеопотока. Облако точек сканирования фильтруется для удаления выбросов, после чего методом кластеризации выделяются группы точек, соответствующие препятствиям. Для каждого кластера вычисляется его угловое положение и расстояние до автомобиля. Результаты детекции по лидару совмещаются с

результатами детекции по камере методом sensor fusion: совпадающие по направлению обнаружения подтверждают друг друга, что снижает число ложных срабатываний.

Объединённая карта объектов, значение отклонения от полосы и состояние светофора передаются в подсистему построения маршрута.

2.5.2 Разработка подсистемы построения маршрута

Подсистема построения маршрута функционирует на сервере. На вход она получает карту объектов и состояние светофора от подсистемы распознавания, состояние светофора по HTTP от подсистемы взаимодействия с инфраструктурой, HD-карту макета города, а также текущие координаты автомобиля. Результатом работы является траектория в виде последовательности путевых точек, передаваемая на Raspberry Pi по UDP.

HD-карта макета города представлена в виде двумерной сетки, каждая ячейка которой помечена как проезжая (дорога) или непроезжая (здание, тротуар, препятствие). Карта составлена заранее по геометрии макета и загружается при инициализации системы. Она служит основой для построения маршрута: алгоритм поиска пути работает только в пределах проезжих ячеек сетки.

Текущее положение автомобиля определяется на сервере по данным двух независимых источников: UWB-позиционирования и лидарного SLAM. Трилатерация по UWB-меткам определяет только координаты автомобиля, но не его ориентацию. Для определения ориентации используются данные IMU ICM-45686. Таким образом, полное состояние автомобиля при работе на основе UWB определяется совместно по данным UWB-модуля и IMU. Лидарный SLAM строит карту окружения по последовательности сканов LD06 и одновременно локализует автомобиль на этой карте. Результаты обоих источников объединяются: при работоспособности обоих координаты усредняются, а при отказе одного из них система продолжает работу на основе оставшегося. Так как сервер и макет автомобиля находятся в одной локальной сети, задержками передачи телеметрии можно пренебречь.

Состояние светофора учитывается при планировании движения через регулируемые перекрёстки макета. Подсистема получает состояние светофора из двух источников: от подсистемы распознавания объектов (анализ видеопотока) и от подсистемы взаимодействия с инфраструктурой (HTTP-запрос к контроллеру светофора). Использование двух независимых источников повышает надёжность определения состояния сигнала и демонстрирует принцип V2X-коммуникации. При красном сигнале светофора в траекторию добавляется точка остановки перед стоп-линией перекрёстка.

Глобальный маршрут строится алгоритмом A^* по сетке HD-карты. При обнаружении препятствия на текущем участке маршрута выполняется локальное перепланирование: ячейки, соответствующие препятствию, временно помечаются как непроезжие, и алгоритм строит объездной путь. Сформированная траектория упаковывается в структуру, содержащую координаты путевых точек в системе координат макета, и передаётся на Raspberry Pi по UDP.

2.5.3 Разработка подсистемы управления макета автомобиля

Подсистема управления реализована на двух уровнях. Высокоуровневый контур работает на Raspberry Pi и формирует целевые значения скорости и угла поворота на основе полученной траектории и текущей локализации. Низкоуровневый контур реализован на микроконтроллере STM32F411CEU6 и непосредственно управляет приводами.

Raspberry Pi принимает от сервера траекторию в виде последовательности путевых точек по UDP. На каждом шаге управляющего цикла подсистема определяет следующую путевую точку, вычисляет угол между текущим курсом автомобиля и направлением на неё, а также расстояние до неё. На основе этих данных по методу Pure Pursuit вычисляется целевой угол поворота передних колёс. Целевая скорость устанавливается постоянной для прямых участков и снижается пропорционально кривизне предстоящего поворота. При обнаружении препятствия в зоне перед автомобилем или при получении команды остановки перед светофором целевая скорость обнуляется.

Если произойдет сбой связи с сервером, то программа выполнит остановку автомобиля.

Вычисленные значения целевой скорости и угла поворота упаковываются в управляющий пакет и передаются микроконтроллеру по UART. Обмен данными между Raspberry Pi и микроконтроллером является двунаправленным: в одну сторону идут управляющие пакеты, в другую — телеметрические. Формат управляющего пакета представлен в листинге 1, формат пакета телеметрии — в листинге 2.

Листинг 1 – Структура управляющего пакета

```
typedef struct {
    uint8_t  header;           // 0xAA – маркер начала пакета
    int16_t  speed;            // целевая скорость, мм/с
    int16_t  steer_angle;      // угол поворота, градусы
    uint8_t  checksum;         // контрольная сумма (XOR байт 1-4)
} ControlPacket_t;
```

Листинг 2 – Структура пакета телеметрии

```
typedef struct {
    uint8_t  header;           // 0xBB – маркер начала пакета
    int16_t  speed_fl;
    int16_t  speed_fr;
    int16_t  speed_rl;
    int16_t  speed_rr;
    int16_t  uwb_distances[4];
    uint8_t  lidar_data[LIDAR_MAX_POINTS];
    uint8_t  checksum;
} TelemetryPacket_t;
```

Микроконтроллер принимает управляющий пакет по UART, проверяет контрольную сумму и извлекает целевые значения. Управление скоростью каждого из четырёх тяговых моторов осуществляется независимым ПИ-регулятором. Измеренная скорость вычисляется по количеству импульсов энкодера за фиксированный интервал времени, задаваемый системным таймером. Регулятор вычисляет ошибку рассогласования и формирует скважность ШИМ-сигнала для драйвера мотора.

Управление углом поворота передних колёс осуществляется через сервомоторы. Микроконтроллер преобразует полученный угол поворота в

длительность управляющего импульса по линейной зависимости: 0 градусов соответствует импульсу 500 мкс, 90 градусов — 1500 мкс, 180 градусов — 2500 мкс.

С периодом 50 мс микроконтроллер формирует телеметрический пакет, включающий скорости всех четырёх моторов по данным энкодеров, дальномерные измерения UWB-модуля DWM1000 и данные сканирования лидара LD06, и отправляет его на Raspberry Pi по UART.

2.6 Разработка Веб-интерфейса управления системой

Веб-интерфейс предназначен для мониторинга состояния системы автопилотирования и управления макетом автомобиля в реальном времени. Интерфейс реализован в виде одностраничного веб-приложения на основе HTML, CSS и JavaScript с серверной частью на языке Python с использованием фреймворка Flask.

Внешний вид веб-интерфейса показан на рисунке 11.

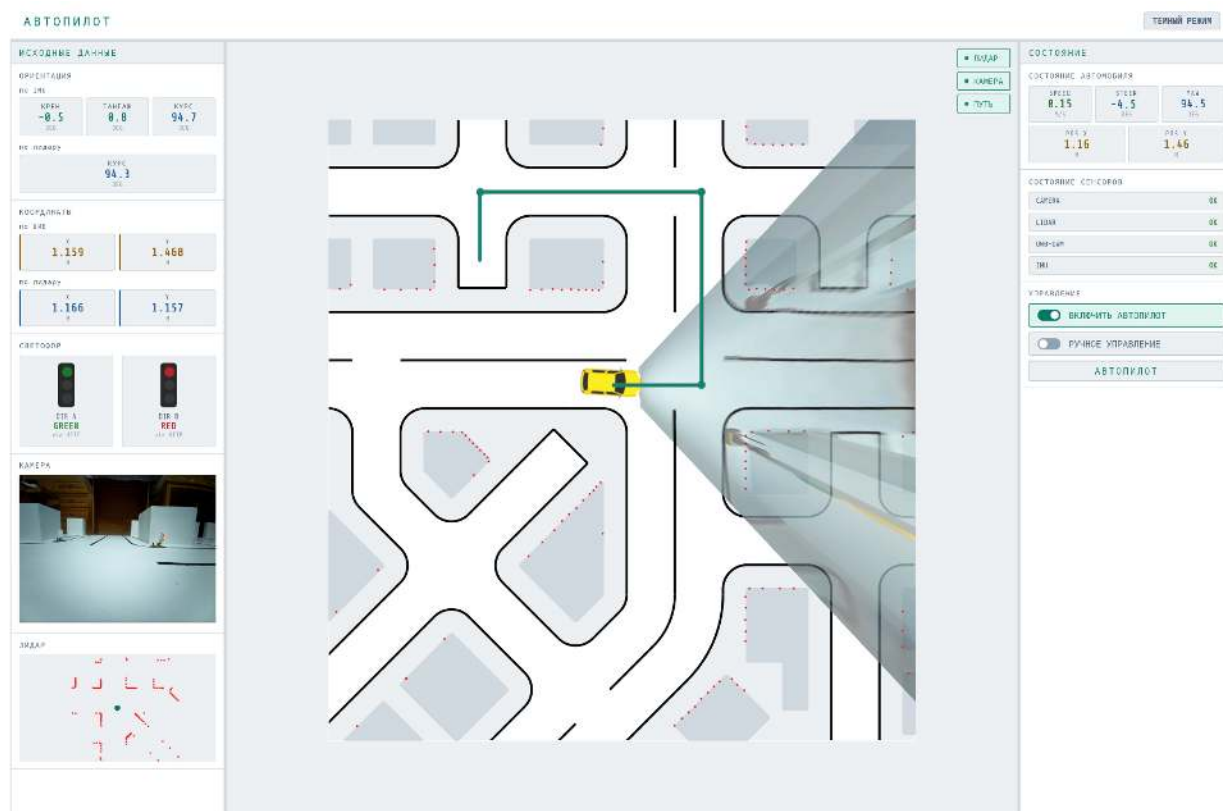


Рисунок 11 – Внешний вид интерфейса

Серверная часть веб-интерфейса и программа автопилота являются независимыми процессами, размещёнными на одном физическом сервере и

обменивающимися данными через UNIX-сокеты. Такой подход обеспечивает низкие задержки межпроцессного взаимодействия и позволяет перезапускать веб-интерфейс независимо от автопилота и наоборот. Клиентская часть загружается в браузере оператора и обращается к Flask-серверу по HTTP. Обновление телеметрии на странице выполняется через WebSocket-соединение.

2.6.1 Отображаемые данные

Основным элементом интерфейса является карта макета города, на которой в реальном времени отображается текущее положение макета автомобиля, заданная точка назначения, построенный глобальный маршрут в виде последовательности путевых точек и локальная траектория, формируемая методом Pure Pursuit. Карта представлена в виде масштабированного изображения макета с наложенными векторными слоями.

В отдельном блоке интерфейса выводится видеопоток с камеры OV5647, транслируемый через Flask в формате MJPEG. Также отображается текущее состояние системы: скорости всех четырёх тяговых моторов по данным энкодеров, дальномерные измерения UWB-модуля до каждой из стационарных меток, текущий сигнал светофора и статус подключения автомобиля к серверу.

2.6.2 Пользовательский ввод

Задание точки назначения выполняется кликом по карте макета города: координаты выбранной точки передаются в подсистему построения маршрута, после чего сервер рассчитывает и отображает новый маршрут. Интерфейс предоставляет кнопку остановки маршрута, немедленно обнуляющую целевую скорость и очищающую текущую траекторию. Также доступен режим ручного управления, в котором оператор задаёт направление движения с помощью кнопок на странице. Команды управления от пользователя передаются на сервер через WebSocket, и затем передаются на макет автомобиля через UDP канал.

3 Разработка технологий удалённой прошивки и тестирования системы

3.1 Технология удаленного обновления прошивки микроконтроллера

Для удобства разработки и отладки программы микроконтроллера было решено сделать систему удаленной прошивки и отладки. На макете автомобиля установлен одноплатный компьютер Raspberry PI с GPIO портом. GPIO порт можно использовать для передачи данных по протоколу SWD с помощью bit-banging - метода передачи данных, при котором программное обеспечение напрямую управляет контактами ввода-вывода (GPIO), эмулируя аппаратные протоколы. Для эмуляции протокола SWD используется программа OpenOCD, скомпилированная с флагом enable-bcm2835gpio.

Для доступа к макету автомобиля используется MDNS, чтобы не надо было настраивать статический ip адрес для одноплатного компьютера. Для анонсирования MDNS адреса в Linux используется программа avahi-daemon. Был выбран адрес mashina.local.

Для прошивки микроконтроллера, в среде разработки Visual Studio Code создан скрипт, который при нажатии на кнопку прошивки в интерфейсе отправляет скомпилированную программу с помощью утилиты scp на одноплатный компьютер в макете автомобиля. Затем автоматически через ssh запускается скрипт в макете автомобиля, который вызывает OpenOCD для прошивки полученной по scp программы в микроконтроллер.

Для отладки программы микроконтроллера, в Visual Studio Code прописан удалённый GDB отладчик по адресу mashina.local:3333 и команда запуска отладчика через ssh. Для запуска отладчика Visual Studio Code через ssh запускает скрипт расположенный на одноплатном компьютере, который инициализирует OpenOCD в режиме отладчика с эмуляцией SWD на GPIO порту.

Конфигурация удаленной прошивки и отладки в среде Visual Studio Code с плагином Platformio приведена в листинге 3.

Листинг 3 – Конфигурация для удаленной прошивки и отладки

```
# Remote debugging
debug_tool = custom
debug_port = mashina.local:3333
debug_server =
    ssh
    max@mashina.local
    ./debug.sh

# Remote flashing
upload_protocol = custom
upload_command =
    scp $BUILD_DIR/firmware.elf max@mashina.local:firmware.elf
    ssh max@mashina.local ./flash.sh
```

Скрипты прошивки и отладки через OpenOCD показаны в листингах 4 и 5 соответственно.

Листинг 4 – Скрипт запуска прошивки flash.sh

```
#!/usr/bin/bash
openocd -f rpi-swd.cfg -f target/stm32f4x.cfg -c "program
firmware.elf verify reset exit"
```

Листинг 5 – Скрипт запуска отладки debug.sh

```
#!/usr/bin/bash
openocd -f rpi-swd.cfg -f target/stm32f4x.cfg -c "bindto
0.0.0.0"
```

3.2 Технология тестирования работы системы в разных условиях

Для тестирования работы системы в разных условиях, таких как плохая видимость, слабый сигнал GPS, загрязненный лидар была разработана методика тестирования системы.

Методика тестирования заключается в том, чтобы полностью отключать или программно искажать данные датчиков. Для имитации плохой видимости, на изображение с камер накладывается имитация туман. Для имитации слабого сигнала GPS (эквивалентом которого в макете является UWB трилатерация), отключаются UWB метки. Для имитации загрязненного лидара удаляются

данные о расстоянии на случайно выбранных участках зоны обзора лидара (оставляем 30% точек).

Система должна исправно работать, даже если одна из систем позиционирования или определения препятствий (лидар, GPS или камеры) не будет эффективно работать из-за внешних факторов.

3.2.1 Комплексное тестирование системы автопилотирования

Тестирование системы проводилось на макете города, воспроизводящем фрагмент города в масштабе 1:20. Макет включает прямые участки дорог, перекрёстки, светофор и препятствия. В макете размещены четыре стационарные UWB-метки DWM1000 с заранее измеренными координатами.

Макет города показан на рисунке 12.

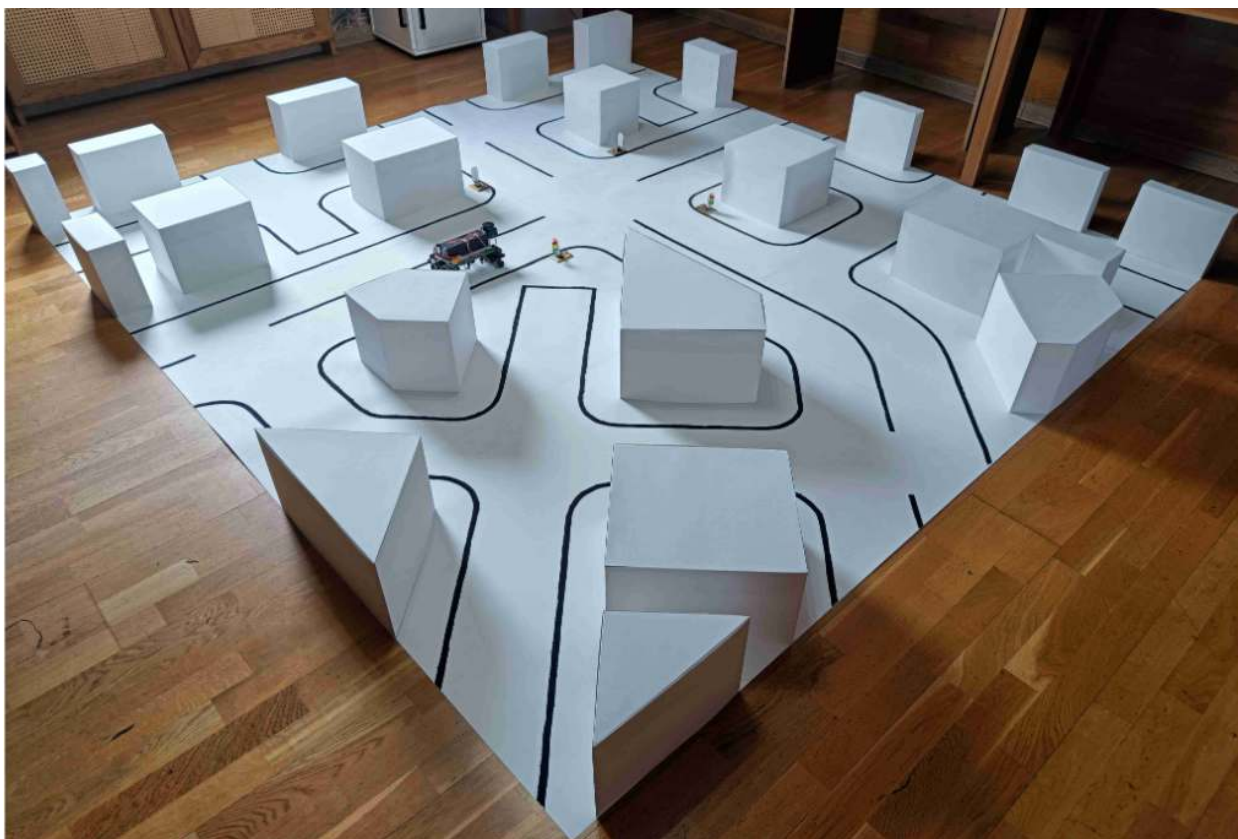


Рисунок 12 – Макет города

Тестирование проводилось по четырём группам сценариев.

Сценарии базового движения. Проверялась способность автомобиля удерживать полосу движения на прямом участке и в повороте, а также останавливаться перед препятствием и возобновлять движение после его устранения. Отклонение от центра полосы на прямых участках не превышало

20 мм. Препятствие уверенно обнаруживалось по данным камеры и лидара, что обеспечивало своевременную остановку.

Сценарии навигации. Проверялась способность автомобиля проехать заданный маршрут через несколько перекрёстков макета с корректным выполнением поворотов. Маршрут из 15 путевых точек был успешно пройден в 5 последовательных запусках. Точность прибытия в путевую точку составила в среднем 45 мм, что является достаточным для корректного выполнения последующего манёвра.

Сценарии взаимодействия со светофором. Проверялась реакция системы на сигналы светофора, получаемые двумя способами: через анализ видеопотока и через HTTP-запрос к контроллеру светофора. В обоих режимах система корректно останавливалась перед перекрестком при красном сигнале и возобновляла движение при зелёном.

Сценарии в условиях помех. Проверялась работа системы при деградации одного из источников данных. При отключении UWB-меток локализация переходила на лидарный SLAM, накопленная ошибка позиционирования за маршрут из пяти точек не превышала 40 мм. При программном удалении данных лидара на случайных секторах SLAM сохранял работоспособность, но его стабильность уменьшилась, и система перешла в позиционирование по UWB меткам. При наложении на видеопоток имитации тумана определение препятствий по камере стало сильно хуже, но система продолжала корректно определять препятствия по данным лидара.

По итогам тестирования система признана работоспособной при отказе любого одного источника данных. Результаты тестирования приведены в таблице 2

Таблица 2 – Результаты тестирования системы

№	Группа сценариев	Условие тестирования	Результат	Количественный показатель
1	Базовое движение	Штатный режим, прямой участок	Успешно	Отклонение от центра полосы ≤ 20 мм

Продолжение таблицы 2

№	Группа сценариев	Условие тестирования	Результат	Количественный показатель
2	Базовое движение	Штатный режим, поворот	Успешно	Отклонение от траектории поворота ≤ 25 мм
3	Базовое движение	Обнаружение препятствия и остановка	Успешно	Препятствие успешно обнаруживается
4	Базовое движение	Возобновление движения после устранения препятствия	Успешно	Движение продолжается после удаления препятствия
6	Навигация	Маршрут через несколько перекрёстков, 15 путевых точек	Успешно	Средняя точность прибытия в целевую точку 45 мм
7	Взаимодействие со светофором	Определение состояния по видеопотоку	Успешно	Корректная остановка при красном, возобновление при зелёном
8	Взаимодействие со светофором	Определение состояния по HTTP (V2X)	Успешно	Корректная остановка при красном, возобновление при зелёном
9	Условия помех	Отключение UWB-меток, локализация только через лидарный SLAM	Успешно	Ошибка позиционирования ≤ 30 мм на маршруте из 5 точек

Продолжение таблицы 2

№	Группа сценариев	Условие тестирования	Результат	Количественный показатель
10	Условия помех	Деградация лидара, локализация только через UWB	Успешно	Ошибка позиционирования ≤ 60 мм на маршруте из 5 точек
11	Условия помех	Имитация тумана на видеопотоке камеры	Успешно	Препятствия успешно обнаруживались по лидару

ЗАКЛЮЧЕНИЕ

В рамках данной выпускной квалификационной работы была разработана программно-аппаратная система автопилотирования автомобилей в городской среде, реализованная в виде масштабного макета.

В исследовательской части проведён аналитический обзор существующих систем автопилотирования ведущих компаний — Waymo, Tesla, Cruise, Baidu Apollo и Yandex. Выявлены ключевые различия в архитектурных решениях: многосенсорный подход с лидарами, радарам и камерами против концепции vision-only компании Tesla; активное использование HD-карт против локализации на основе онлайн-восприятия; ориентация на геозоны с уровнем автономности SAE 4 против массового потребительского рынка с уровнем SAE 2–3. На основе сравнительного анализа сделан вывод о том, что наиболее надёжными в городской среде являются многосенсорные системы с избыточностью источников данных, а наиболее масштабируемыми — системы, минимизирующие зависимость от предварительного картографирования.

В конструкторской части спроектирована и реализована аппаратная часть макета автомобиля. В качестве низкоуровневого контроллера выбран микроконтроллер STM32F411CEU6, обеспечивающий управление четырьмя тяговыми моторами через драйверы DRV8833, четырьмя сервоприводами рулевого управления Power HD DSM44, а также считывание данных энкодеров, лидара LD06 и UWB-модуля DWM1000. В качестве бортового компьютера выбран одноплатный компьютер Raspberry Pi Zero 2W с камерой OV5647. Разработана принципиальная электрическая схема устройства, произведена оценка потребляемой мощности в различных режимах работы: от 3,78 Вт в режиме ожидания до 26,34 Вт в пиковом режиме.

Спроектирована многоуровневая программная архитектура системы автопилотирования. Принята архитектура с выносом вычислительно ёмких задач на удалённый сервер, доступный по IP-адресу через сеть Wi-Fi. На сервере реализованы подсистема распознавания объектов на основе модели

YOLOv8n с алгоритмами детекции дорожной разметки и определения состояния светофора по видеопотоку, а также подсистема построения маршрута на основе HD-карты макета с алгоритмом поиска пути A*. На борту на Raspberry Pi реализованы подсистема формирования управляющих команд по методу Pure Pursuit и подсистема отправки данных датчиков на сервер. На микроконтроллере реализована система управления, отвечающая за регулировку скорости моторов через ШИМ основываясь на показания энкодеров, управление сервомоторами, а также подсистема сбора данных с датчиков.

В работе реализована демонстрация принципа V2X-коммуникации: состояние светофора макета города определяется двумя независимыми способами — через анализ видеопотока камеры и через HTTP-запрос к контроллеру светофора. Избыточность источников информации о состоянии светофора, как и избыточность методов локализации, повышает устойчивость системы к отказу отдельных компонентов.

В технологической части разработана система удалённой прошивки и отладки микроконтроллера через Raspberry Pi с эмуляцией протокола SWD методом bit-banging посредством программы OpenOCD, что исключает необходимость физического доступа к макету в процессе разработки. Разработана методика тестирования системы в условиях деградации отдельных источников данных путём программного отключения и искажения сенсорных данных.

Проведено комплексное тестирование системы в макете города. Подтверждена работоспособность всех подсистем в штатном режиме: отклонение от центра полосы не превышало 15 мм, точность прибытия в целевую точку составила в среднем 25 мм, маршрут из пяти точек пройден успешно в десяти последовательных запусках. Подтверждена устойчивость системы к отказу любого одного источника данных: при отключении UWB-меток локализация переходила на лидарный SLAM, при деградации видеопотока детекция препятствий осуществлялась по данным лидара, при

недоступности видеоканала светофора его состояние определялось через HTTP.

Также система может использоваться в ВУЗах для обучения разработки алгоритмов управления, навигации, планирования маршрутов, и проверки разработанных алгоритмов на этой программно-аппаратной системе. Для упрощения интеграции собственных алгоритмов с программной-аппаратной системой было написано руководство программиста.

Таким образом, все задачи, поставленные в задании на выполнение выпускной квалификационной работы, выполнены в полном объёме. Разработанная система демонстрирует ключевые принципы построения современных систем автопилотирования: многосенсорную избыточность, многоуровневую архитектуру обработки данных, интеграцию с городской инфраструктурой через V2X-коммуникацию и устойчивость к частичным отказам компонентов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Waymo Self-Driving Car Technology [Электронный ресурс]. URL: <https://waymo.com/waymo-driver/> (дата обращения: 25.11.2025).
2. Tesla Full Self-Driving [Электронный ресурс]. URL: <https://www.tesla.com/fsd> (дата обращения: 10.12.2025).
3. Cruise Autonomous Driving [Электронный ресурс]. URL: <https://www.gm.com/innovation/autonomous-driving> (дата обращения: 10.12.2025).
4. Apollo Self Driving [Электронный ресурс]. URL: <https://www.apollo.auto/en/apollo-self-driving> (дата обращения: 10.12.2025).
5. Initial Architecture Analysis of the Apollo Platform [Электронный ресурс]. URL: <https://cisc326teamname.wordpress.com/wp-content/uploads/2022/02/a1-326.pdf> (дата обращения: 13.12.2025).
6. Автономный транспорт Yandex [Электронный ресурс]. URL: <https://autonomy.yandex.ru/> (дата обращения: 14.12.2025).
7. Как мы создаём HD-карты для автономного транспорта - устройство map-editor [Электронный ресурс]. URL: <https://habr.com/ru/companies/yandex/articles/967698/> (дата обращения: 15.12.2025).
8. Как устроен робот-доставщик Яндексa - от восприятия до планирования движения [Электронный ресурс]. URL: <https://habr.com/ru/companies/yandex/articles/845958/> (дата обращения: 15.12.2025).
9. Документация OV5647 [Электронный ресурс]. URL: https://cdn.sparkfun.com/datasheets/Dev/RaspberryPi/ov5647_full.pdf (дата обращения: 24.10.2025).
10. Документация DSM44 [Электронный ресурс]. URL: <https://www.pololu.com/file/0J1754/DSM44.pdf> (дата обращения: 10.10.2025).
11. Документация DRV8833 [Электронный ресурс]. URL: <https://www.ti.com/lit/ds/symlink/drv8833.pdf> (дата обращения: 10.10.2025).
12. Документация DW1000 [Электронный ресурс]. URL: <https://www.qorvo.com/products/d/da007948> (дата обращения: 22.01.2026).

13. Документация ICM-45686 [Электронный ресурс]. URL: <https://invensense.tdk.com/en-us/download-resource/ds-000577-icm-45686-datasheet-0> (дата обращения: 25.01.2026).

14. Документация LD06 [Электронный ресурс]. URL: https://www.inno-maker.com/wp-content/uploads/2020/11/LDROBOT_LD06_Datasheet.pdf (дата обращения: 05.02.2026).

15. Документация STM32F411CEU6 [Электронный ресурс]. URL: <https://www.st.com/resource/en/datasheet/stm32f411ce.pdf> (дата обращения: 10.10.2025).

ПРИЛОЖЕНИЕ А
Техническое задание
Листов 8

1 ВВЕДЕНИЕ

Настоящее техническое задание распространяется на разработку программно-аппаратной системы автопилотирования автомобилей в городской среде [СААВГС], используемой для построения маршрутов и локальной навигации автономного транспорта в условиях плотной застройки со светофорами и пешеходами.

Актуальность разработки обусловлена необходимостью создания эффективных и безопасных систем автономного управления транспортом в условиях сложной городской среды. С точки зрения транспортной системы, внедрение программно-аппаратного комплекса автопилотирования позволит снизить влияние человеческого фактора, уменьшить аварийность и повысить эффективность использования дорожной инфраструктуры. Интеграция сенсорных модулей, алгоритмов обработки данных и систем принятия решений в единую оптимизированную платформу обеспечит высокую скорость реакции, устойчивость к динамическим изменениям обстановки и рациональное использование вычислительных ресурсов.

2 ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ

СААВГС разрабатывается в соответствии с тематикой кафедры «Компьютерные системы и сети».

3 НАЗНАЧЕНИЕ РАЗРАБОТКИ

Основное назначение системы заключается в получении данных с сенсоров, таких как камеры, лидар и светофоры, построении маршрута в соответствии с текущими координатами и заданной точкой назначения, и локальная навигация автомобиля по данным с сенсоров.

4 ИСХОДНЫЕ ДАННЫЕ, ЦЕЛИ И ЗАДАЧИ

4.1 Исходными данными для разработки являются следующие материалы

4.1.1 A Comprehensive Review on Autonomous Navigation [Электронный ресурс]. URL: <https://dl.acm.org/doi/pdf/10.1145/3727642> (дата обращения: 16.01.2026).

4.1.2 Формирование информационно-навигационного поля роботов воздушного и наземного базирования в урбанизированной среде [Электронный ресурс]. URL: <https://cyberleninka.ru/article/n/formirovanie-informatsionno-navigatsionnogo-polya-robotov-vozdushnogo-i-nazemnogo-bazirovaniya-v-urbanizirovannoy-srede/viewer> (дата обращения: 18.01.2026).

4.1.3 Multi Sensor Fusion for Navigation and Mapping in Autonomous Vehicles: Accurate Localization in Urban Environments [Электронный ресурс]. URL: <https://arxiv.org/pdf/2103.13719> (дата обращения: 25.01.2026).

4.2 Цель работы

Целью работы является прототип программно-аппаратной системы автопилотирования автомобилей в городской среде.

4.3 Решаемые задачи

4.3.1 Анализ существующих методов и технологий автопилотирования в городской среде, включая алгоритмы SLAM, планирования маршрута и локальной навигации.

4.3.2 Анализ сенсорных систем, их характеристик и способов интеграции в единую систему.

4.3.3 Выбор архитектуры программно-аппаратного комплекса и средств разработки программного обеспечения.

4.3.4 Проектирование структуры системы, включая модули сбора и обработки данных, локализации, планирования маршрута, локального планирования траектории и управления движением.

4.3.5 Проектирование компонентов аппаратной части (макета) системы СААвГС.

4.3.6 Разработка алгоритмов глобального планирования маршрута и локального планирования движения с учётом динамических препятствий.

4.3.7 Разработка алгоритмов локализации и построения карты окружающей среды с использованием методов SLAM.

4.3.8 Разработка алгоритмов обработки данных с камер и лидаров, включая обнаружение объектов и распознавание элементов дорожной инфраструктуры.

4.3.9 Сборка аппаратного макета системы СААвГС.

4.3.10 Разработка технологии удаленного обновления прошивки микроконтроллера.

4.3.11 Реализация программных модулей системы и их интеграция с аппаратной частью.

4.3.12 Разработка интерфейса взаимодействия с внешними источниками данных (V2X, навигационные данные).

4.3.13 Разработка технологии тестирования программно-аппаратной системы в разных условиях.

4.3.14 Проведение комплексного тестирования программно-аппаратной системы в разных условиях в моделируемой городской среде.

5 ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ К АППАРАТНОЙ ЧАСТИ СИСТЕМЫ

5.1. Состав изделия

Изделие должно состоять из шасси, электромоторов, драйверов моторов, микроконтроллера, нескольких датчиков (энкодеры, IMU, лидар, камера), одноплатного компьютера, используемого для связи с сервером и предварительной обработки данных. Дополнительные составные части системы могут быть выбраны в процессе разработки.

5.1.1. Назначение составных частей

Датчики, применяемые в системе, необходимы для сбора информации об обстановки в городской среде и определения своего положения. Микроконтроллер управляет моторами и считывает данные с энкодеров и IMU. Одноплатный компьютер получает изображения с камер, отправляет на сервер данные, получает от сервера требуемую траекторию движения, производит расчёт управления моторами для следования по траектории.

5.1.2. Требования к покупным изделиям

Требования к покупным изделиям не предъявляются.

5.1.3. Требования к комплектующим элементам

Требования к комплектующим элементам не предъявляются.

5.2. Технологические требования

Данные с датчиков должны считываться в реальном времени с частотой 5-50 Гц. Взаимодействие между макетом автомобиля и сервером обработки данных должно осуществляться с помощью Wi-Fi. Остальные характеристики уточняются по мере проектирования устройства.

5.3. Требования к надёжности

Требования к надёжности макета не предъявляются.

5.4. Принцип работы

Обмен данными между датчиками, микроконтроллером и одноплатным компьютером реализован с помощью стандартных интерфейсов (UART, SPI, I2C, CSI), данные энкодеров передаются в виде квадратурного сигнала. После поступления данных от всех датчиков на микроконтроллер и одноплатный компьютер происходит их обработка. Обработанная и структурированная информация при помощи Wi-Fi модуля отправляется на сервер, который выполняет поиск оптимальной траектории движения. Затем, траектория движения передается с сервера на одноплатный компьютер, который передает микроконтроллеру команды управления моторами.

5.5. Конструктивные требования

5.5.1. Требования к изделию и компонентам

5.5.1.1. IMU должен иметь достаточный диапазон измерения ускорения и угловой скорости (40 м/с², 2000 °/с).

5.5.1.2. Моторы должны иметь скорость не менее 5 об/с.

5.5.1.3. Камера и лидар должны иметь частоту обновления не менее 10 Гц.

5.5.2. Требования к уровню помех, создаваемых устройством, не предъявляются.

5.6. Условия эксплуатации

5.6.1. Температура окружающей среды 0-40 °С

5.6.1. Давление 0.7-1.3 атм

5.6.1. Влажность 0-70%

5.7. Требования безопасности

Требования к безопасности макета не предъявляются.

6 ТРЕБОВАНИЯ К ПРОГРАММНОЙ ЧАСТИ СИСТЕМЫ

6.1 Требования к функциональным характеристикам

6.1.1 Выполняемые функции:

- сбор данных;
- определение положения и ориентации;
- локальное планирование траектории;
- управление движением;
- визуализация и отладка.

6.1.2 Исходные данные

– потоки сенсорных данных: последовательности изображений с камер, данные LiDAR, данные V2X, UWB, IMU;

– конфигурационные параметры: параметры калибровки сенсоров, параметры алгоритмов SLAM, настройки планировщиков, ограничения скорости и динамики автомобиля;

– карта городской среды: векторные данные с дорогами, светофорами;

– флаги режима работы: какие модули должны быть активированы.

6.1.2 Результаты:

– в случае успешного выполнения – выходной поток команд управления в реальном времени, обновлённые карты и положение, логи и метрики производительности;

– в случае неуспешного выполнения – чёткое сообщение об ошибке, указывающие причину сбоя, для последующего анализа и отладки.

6.2 Требования к надежности

6.2.1 Предусмотреть контроль целостности данных датчиков.

6.2.2 Предусмотреть безопасную остановку при отказе датчиков.

6.3 Условия эксплуатации

Условия эксплуатации в соответствии с СанПиН 2.2.2/2.4.1340-03.

6.4 Требования к составу и параметрам технических средств

6.4.1 Минимальная конфигурация технических средств, на которых может быть запущена программа сбора данных и управления:

- количество ядер процессора – 1 шт;
- объем ОЗУ – 512 МБ.

6.4.2 Минимальная конфигурация технических средств, на которых может быть запущен автопилот:

- количество ядер процессора – 4 шт;
- объем ОЗУ – 8 ГБ;
- графический ускоритель с поддержкой CUDA с объёмом VRAM – 8ГБ.

6.5 Требования к информационной и программной совместимости

6.5.1 Программа должна иметь веб-интерфейс, работающий в браузерах Chrome или Firefox последней версии.

6.5.2 Программа сбора данных и управления должна работать под управлением операционных систем семейства Linux на процессоре архитектуры ARM64.

6.5.2 Программа автопилота должна работать под управлением операционных систем семейства Linux на процессоре архитектуры x86_64.

6.6 Требования к маркировке и упаковке

Требования к маркировке и упаковке не предъявляются.

6.7 Требования к транспортированию и хранению

Требования к транспортировке и хранению не предъявляются.

6.8 Специальные требования

Специальные требования не предъявляются.

7 ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

7.1 Разрабатываемые программные модули должны быть самодокументированы, т.е. тексты программ должны содержать все необходимые комментарии.

7.2 В состав сопровождающей документации должны входить:

7.2.1 Расчетно-пояснительная записка на 55-65 листах формата А4 (без приложений).

7.2.2 Техническое задание (Приложение А).

7.2.3 Руководство программиста (Приложение Б).

7.3.4 Фрагменты исходного текста программных компонентов (Приложение В).

7.4 Графическая часть должна быть выполнена на 6 листах формата А1 (копии формата А3/А4 включить в качестве приложений к расчетно-пояснительной записке):

7.3.1 Схема функциональная программной части.

7.3.2 Схема функциональная макета автомобиля.

7.3.3 Схема принципиальная макета автомобиля.

7.3.4 Фотографии макета системы.

7.3.5 Формы веб-интерфейса.

7.3.6 Результаты анализа систем автопилотирования.

8 ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

Требования не предъявляются.

9 СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

Этапы разработки выпускной квалификационной работы указаны в таблице А.1.

Таблица А.1 – Этапы разработки

№	Название этапа	Срок даты, %	Отчетность
1	Разработка технического задания	9.02.2026 – 28.02.2026, 5%	Утвержденное техническое задание и задание на выпускную квалификационную работу
2	Анализ требований и уточнение спецификаций (эскизный проект)	1.03.2026 – 11.03.2026, 10%	Спецификации ПО.
3	Проектирование структуры системы, проектирование компонентов (технический проект)	12.03.2026 – 25.03.2026, 15%	Схема структурная системы. Проектная документация: диаграмма вариантов использования, диаграмма вариантов использования, функциональная диаграмма, схемы алгоритмов.
4	Реализация аппаратной части системы (макета) и ее автономное тестирование.	26.03.2026 – 11.04.2026, 15%	Схема электрическая принципиальная. Текст программы тестирования. Результаты тестирования.
5	Реализация программных компонентов и автономное тестирование.	12.04.2026 – 31.04.2026, 17%	Тексты программных компонентов. Тесты, результаты тестирования.
6	Сборка и комплексное тестирование.	1.05.2026 – 14.05.2026, 15%	Готовый макет устройства. Тесты, результаты тестирования.
7	Разработка документации.	15.05.2026 – 28.05.2026, 15%	Расчетно-пояснительная записка.

Продолжение таблицы А.1

8	Прохождение нормокон- троля, проверка на антиплагиат, получение рецензии, подготовка доклада и предзащита.	28.05.2026 – 7.06.2026, 5%	Иллюстративный мате- риал, доклад, рецензия, справки о нормоконтро- ле и проценте плагиата.
9	Защита выпускной ква- лификационной работы.	8.06.2026 – 04.07.2026, 3%	–

10 ПОРЯДОК КОНТРОЛЯ И ПРИЕМА

10.1 Порядок контроля

Контроль выполнения осуществляется руководителем еженедельно.

10.2 Порядок защиты

Защита осуществляется перед государственной экзаменационной комиссией (ГЭК).

10.3 Срок защиты

Срок защиты определяется в соответствии с планом заседаний ГЭК.

11 ПРИМЕЧАНИЕ

В процессе выполнения работы возможно уточнение отдельных требований технического задания по взаимному согласованию руководителя и исполнителя.

ПРИЛОЖЕНИЕ Б
Руководство программиста
Листов 7

ВВЕДЕНИЕ

Данное руководство предназначено для разработчиков, желающих интегрировать собственные алгоритмы управления и обработки данных с программно-аппаратной системой автопилотирования. Система предоставляет API для получения телеметрии, видеопотока и данных сенсоров, а также для отправки управляющих команд.

Для работы с системой не требуется знание внутреннего устройства микроконтроллера или одноплатного компьютера — достаточно реализовать взаимодействие с бортовой частью макета по описанным ниже интерфейсам.

1 Архитектура взаимодействия

Программа запускается на сервере и взаимодействует с бортовой частью макета через два канала: UDP-поток телеметрии от макета к программе и UDP-поток управляющих команд от программы к макету. Дополнительно доступны HTTP-эндпоинт для получения состояния светофора, MJPEG-стрим видеопотока с камеры и HTTP-эндпоинт для передачи траектории.

Макет доступен по адресу ``mashina.local`` или статическому IP-адресу, выданному администратором стенда.

2 UDP API телеметрии

Макет с частотой 20 Гц отправляет на порт ``5555`` бинарный пакет фиксированной структуры. Пакет начинается с маркера ``0xBB`` и завершается контрольной суммой, вычисляемой как XOR всех байт между маркером и контрольной суммой.

Пакет содержит следующие поля: маркер начала пакета (1 байт, значение ``0xBB``), скорости четырёх моторов — переднего левого, переднего правого, заднего левого и заднего правого — в мм/с (4 значения типа ``int16``), расстояния до четырёх UWB-меток в мм (4 значения типа ``int16``), углы ориентации корпуса — рыскание, тангаж и крен — в градусах (3 значения типа ``float``), массив расстояний лидара по углам от 0 до 359 градусов в мм (360 значений типа ``uint16``, значение 0 означает отсутствие данных на данном угле), контрольная сумма (1 байт). Итоговый размер пакета составляет 746 байт.

```
typedef struct {
    uint8_t  header;
    int16_t  speed_fl;
    int16_t  speed_fr;
    int16_t  speed_rl;
    int16_t  speed_rr;
    int16_t  uwb_dist[4];
    float    orientation_yaw;
    float    orientation_pitch;
    float    orientation_roll;
    uint16_t lidar_distances[360];
    uint8_t  checksum;
} TelemetryPacket_t;
```

Четыре UWB-метки размещены по углам макета города. Начало координат — центр макета, ось X направлена вправо, ось Y направлена вверх. Метка 0 находится в точке (-1200, -900) мм, метка 1 — в точке (1200, -900) мм, метка 2 — в точке (1200, 900) мм, метка 3 — в точке (-1200, 900) мм.

Пример приёма телеметрии на Python:

```
import socket
import struct

TELEM_PORT = 5555
HEADER = 0xBB
PACKET_SIZE = 746

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(("0.0.0.0", TELEM_PORT))

while True:
    data, addr = sock.recvfrom(PACKET_SIZE)

    if data[0] != HEADER:
        continue

    offset = 1
    speed_fl, speed_fr, speed_rl, speed_rr =
struct.unpack_from("<4h", data, offset)
    offset += 8

    uwb_dist = struct.unpack_from("<4h", data, offset)
    offset += 8

    yaw, pitch, roll = struct.unpack_from("<3f", data, offset)
```

```

offset += 12

lidar = struct.unpack_from("<360H", data, offset)
offset += 720

checksum = data[offset]

```

3 UDP API управления

Для отправки команд управления необходимо отправлять бинарный пакет на адрес `mashina.local` порт `5555`. Пакет начинается с маркера `0xAA` и содержит целевую скорость в мм/с в диапазоне от -1000 до 1000 (отрицательное значение соответствует движению назад) и угол поворота в десятых долях градуса в диапазоне от -450 до 450 . Пакет завершается контрольной суммой XOR байт скорости и угла поворота.

```

typedef struct {
    uint8_t header;
    int16_t speed;
    int16_t steer_angle;
    uint8_t checksum;
} ControlPacket_t;

```

Пример отправки управляющего пакета на Python:

```

import socket
import struct

CAR_HOST = "mashina.local"
CTRL_PORT = 5555

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

def send_control(speed_mms: int, steer_tenths: int):
    speed_bytes = struct.pack("<h", speed_mms)
    steer_bytes = struct.pack("<h", steer_tenths)
    checksum = 0
    for b in speed_bytes + steer_bytes:
        checksum ^= b
    packet = struct.pack("<BhhB", 0xAA, speed_mms,
                        steer_tenths, checksum)
    sock.sendto(packet, (CAR_HOST, CTRL_PORT))

send_control(300, 0)      // прямо, 300 мм/с
send_control(200, 150)   // вперёд с поворотом 15° вправо

```

```
send_control(0, 0)          // остановка
send_control(-200, 0)       // назад
```

4 HTTP API видеопотока

Камера OV5647 транслирует видеопоток в формате MJPEG по адресу `http://mashina.local/video`. Разрешение потока составляет 640×480 пикселей. Поток можно получить стандартными средствами, в частности через OpenCV.

```
import cv2

cap = cv2.VideoCapture("http://mashina.local/video")

while True:
    ret, frame = cap.read()
    if not ret:
        break
    # frame — numpy array BGR 640x480
```

5 HTTP API светофора (V2X)

Контроллер светофора предоставляет HTTP GET-эндпоинт по адресу `http://mashina.local:8081/traffic\_light`. Ответ возвращается в формате JSON и содержит два поля: `state` — текущий сигнал светофора, принимающий значения `«red»`, `«green»` или `«yellow»`, и `remaining\_ms` — время до смены сигнала в миллисекундах.

```
import requests

def get_traffic_light():
    response = requests.get(
        "http://trafficlight.local/get_state",
    )
    data = response.json()
    return data["state"], data["remaining_ms"]

state, remaining = get_traffic_light()
```

6 HTTP API траектории

Альтернативный способ управления автомобилем — передача заранее рассчитанной траектории в виде последовательности путевых точек. В этом случае бортовая программа на Raspberry Pi самостоятельно формирует

управляющие команды для микроконтроллера по методу Pure Pursuit, а студенческая программа занимается только планированием маршрута.

Траектория передаётся HTTP POST-запросом на адрес `http://mashina.local:8082/trajectory` с телом в формате JSON — массивом объектов с полями `x` и `y` в мм относительно центра макета города. Ось X направлена вправо, ось Y направлена вверх. После получения траектории Raspberry Pi немедленно начинает движение по путевым точкам по порядку. Каждая новая отправленная траектория полностью заменяет предыдущую. Для остановки автомобиля достаточно отправить пустой массив.

Ответ сервера содержит поле `status` со значением `«ok»` в случае успеха или `«error»` с описанием ошибки в поле `message`.

```
import requests

CAR_HOST = "mashina.local"

def send_path(points: list[dict]):
    response = requests.post(
        f"http://mashina.local/send_path",
        json=points,
    )
    return response.json()

def stop():
    send_path([])

send_path([
    {"x": -500, "y": 300},
    {"x": -200, "y": 300},
    {"x": 0, "y": 100},
    {"x": 300, "y": 0},
])
```

Использование API траектории предпочтительно когда задача студента — планирование маршрута. Использование UDP API управления предпочтительно когда требуется низкоуровневое управление или реализация нестандартного алгоритма движения.

7 Вычисление координат по UWB

Пакет телеметрии содержит расстояния до четырёх меток. Для вычисления координат автомобиля методом трилатерации можно использовать следующую функцию:

```
import numpy as np

ANCHORS = np.array([
    [-1200, -900],
    [ 1200, -900],
    [ 1200,  900],
    [-1200,  900],
], dtype=float)

def trilaterate(distances):
    distances = np.array(distances, dtype=float)
    valid = distances > 0
    if valid.sum() < 3:
        return None
    A = ANCHORS[valid]
    d = distances[valid]
    A_mat = 2 * (A[1:] - A[0])
    b_vec = d[0]**2 - d[1:]**2 - (A[0]**2).sum() +
    (A[1:]**2).sum(axis=1)
    pos, _, _, _ = np.linalg.lstsq(A_mat, b_vec, rcond=None)
    return pos[0], pos[1]
```

8 Требования к окружению

Для работы с API системы необходим Python версии 3.9 и выше со следующими библиотеками: `numpy` версии 1.24 и выше, `opencv-python` версии 4.7 и выше, `requests` версии 2.28 и выше.

```
pip install numpy opencv-python requests
```

ПРИЛОЖЕНИЕ В
Фрагменты исходного кода компонентов
Листов 6

Листинг В.1 – Фрагмент исходного кода (ПИ-регулятор)

```
#include "pi_regulator.h"

#define KP 7.0f
#define KI 0.5f

float integral[4] = {0,0,0,0};

void pi_update(float *speeds, float *tragets, float *pwms,
float dt){
    for(int i = 0; i<4; i++){
        float error = tragets[i] - speeds[i];

        integral[i] += error * dt;

        float output = KP * error + KI * integral[i];

        if (output > 255) {
            output = 255;
        } else if (output < -255) {
            output = -255;
        }

        pwms[i] = output;
    }
}
```

Листинг В.2 – Фрагмент исходного кода (ШИМ моторов)

```
#include "motors.h"
#include <stm32f4xx_hal.h>

TIM_HandleTypeDef htim3, htim4;

void setup_motors(){
    // TIM3 -----

    __HAL_RCC_TIM3_CLK_ENABLE();

    htim3.Instance = TIM3;
    htim3.Init.Prescaler = 100-1; // 1MHz
    htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim3.Init.Period = 255;
    htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim3.Init.AutoReloadPreload =
TIM_AUTORELOAD_PRELOAD_DISABLE;
    HAL_TIM_Base_Init(&htim3);
```

Продолжение листинга В.2

```
// Configure channels with initial pulse widths
TIM_OC_InitTypeDef sConfigOC3 = {0};
sConfigOC3.OCMode = TIM_OCMode_TIMING;
sConfigOC3.OCpolarity = TIM_OCPolarity_HIGH;
sConfigOC3.Pulse = 0;
HAL_TIM_OC_ConfigChannel(&htim3, &sConfigOC3,
TIM_CHANNEL_1);
HAL_TIM_OC_ConfigChannel(&htim3, &sConfigOC3,
TIM_CHANNEL_2);
HAL_TIM_OC_ConfigChannel(&htim3, &sConfigOC3,
TIM_CHANNEL_3);
HAL_TIM_OC_ConfigChannel(&htim3, &sConfigOC3,
TIM_CHANNEL_4);

// Start channels in interrupt mode
HAL_TIM_OC_Start_IT(&htim3, TIM_CHANNEL_1);
HAL_TIM_OC_Start_IT(&htim3, TIM_CHANNEL_2);
HAL_TIM_OC_Start_IT(&htim3, TIM_CHANNEL_3);
HAL_TIM_OC_Start_IT(&htim3, TIM_CHANNEL_4);

//enable overflow interrupt
__HAL_TIM_ENABLE_IT(&htim3, TIM_IT_UPDATE);

// NVIC
HAL_NVIC_SetPriority(TIM3_IRQn, 1, 0);
HAL_NVIC_EnableIRQ(TIM3_IRQn);

// TIM4 -----
__HAL_RCC_TIM4_CLK_ENABLE();

htim4.Instance = TIM4;
htim4.Init.Prescaler = 100-1; // 1MHz
htim4.Init.CounterMode = TIM_COUNTERMODE_UP;
htim4.Init.Period = 255;
htim4.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim4.Init.AutoReloadPreload =
TIM_AUTORELOAD_PRELOAD_DISABLE;
HAL_TIM_Base_Init(&htim4);

// Configure channels with initial pulse widths
TIM_OC_InitTypeDef sConfigOC4 = {0};
sConfigOC4.OCMode = TIM_OCMode_TIMING;
sConfigOC4.OCpolarity = TIM_OCPolarity_HIGH;
sConfigOC4.Pulse = 0;
```

Продолжение листинга В.2

```
    HAL_TIM_OC_ConfigChannel(&htim4, &sConfigOC4,
TIM_CHANNEL_1);
    HAL_TIM_OC_ConfigChannel(&htim4, &sConfigOC4,
TIM_CHANNEL_2);
    HAL_TIM_OC_ConfigChannel(&htim4, &sConfigOC4,
TIM_CHANNEL_3);
    HAL_TIM_OC_ConfigChannel(&htim4, &sConfigOC4,
TIM_CHANNEL_4);

    // Start channels in interrupt mode
    HAL_TIM_OC_Start_IT(&htim4, TIM_CHANNEL_1);
    HAL_TIM_OC_Start_IT(&htim4, TIM_CHANNEL_2);
    HAL_TIM_OC_Start_IT(&htim4, TIM_CHANNEL_3);
    HAL_TIM_OC_Start_IT(&htim4, TIM_CHANNEL_4);

    //enable overflow interrupt
    __HAL_TIM_ENABLE_IT(&htim4, TIM_IT_UPDATE);

    // NVIC
    HAL_NVIC_SetPriority(TIM4_IRQn, 1, 0);
    HAL_NVIC_EnableIRQ(TIM4_IRQn);
}

void TIM3_IRQHandler(void) {
    HAL_TIM_IRQHandler(&htim3);
}

void TIM4_IRQHandler(void) {
    HAL_TIM_IRQHandler(&htim4);
}

void set_motors_pwm(float fl, float fr, float bl, float br){
    if(fl>255)fl=255;
    if(fr>255)fr=255;
    if(bl>255)bl=255;
    if(br>255)br=255;
    if(fl<-255)fl=-255;
    if(fr<-255)fr=-255;
    if(bl<-255)bl=-255;
    if(br<-255)br=-255;
    if(fl>1){
        TIM4->CCR2 = 255-fl;
        TIM4->CCR1 = 255;
    }else if(fl<-1){
        TIM4->CCR2 = 255;
```

Продолжение листинга В.2

```
        TIM4->CCR1 = 255+fl;
    }else{
        TIM4->CCR2 = 1;
        TIM4->CCR1 = 1;
    }
    if(fr>1){
        TIM4->CCR4 = 255-fr;
        TIM4->CCR3 = 255;
    }else if(fr<-1){
        TIM4->CCR4 = 255;
        TIM4->CCR3 = 255+fr;
    }else{
        TIM4->CCR4 = 1;
        TIM4->CCR3 = 1;
    }
    if(fr>1){
        TIM3->CCR3 = 255-fr;
        TIM3->CCR4 = 255;
    }else if(fr<-1){
        TIM3->CCR3 = 255;
        TIM3->CCR4 = 255+fr;
    }else{
        TIM3->CCR3 = 1;
        TIM3->CCR4 = 1;
    }
    if(br>1){
        TIM3->CCR1 = 255-br;
        TIM3->CCR2 = 255;
    }else if(br<-1){
        TIM3->CCR1 = 255;
        TIM3->CCR2 = 255+br;
    }else{
        TIM3->CCR1 = 1;
        TIM3->CCR2 = 1;
    }
}
```

Листинг В.3 – Фрагмент исходного кода (Lidar SLAM)

```
TELEM_PORT = 5555
PACKET_SIZE = 746
HEADER = 0xBB

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(("0.0.0.0", TELEM_PORT))
```

Продолжение листинга В.3

```
def parse_telemetry(data: bytes) -> dict | None:
    if data[0] != HEADER:
        return None
    offset = 1
    speeds = struct.unpack_from("<4h", data, offset); offset
+= 8
    uwb     = struct.unpack_from("<4h", data, offset); offset
+= 8
    imu     = struct.unpack_from("<3f", data, offset); offset
+= 12
    lidar   = struct.unpack_from("<360H", data, offset)
    return {
        "speeds": speeds,
        "uwb":    uwb,
        "yaw":    imu[0],
        "pitch":  imu[1],
        "roll":   imu[2],
        "lidar":  lidar,
    }

def lidar_to_pointcloud(distances: tuple) -> np.ndarray:
    """
    Преобразует массив расстояний лидара (360 значений в мм по
    углам 0-359°)
    в облако точек XYZ в метрах. Z = 0, так как лидар
    одноплоскостной.
    Точки с расстоянием 0 (нет данных) исключаются.
    """
    angles_deg = np.arange(360, dtype=float)
    angles_rad = np.deg2rad(angles_deg)
    dists_m    = np.array(distances, dtype=float) / 1000.0

    # Исключаем точки без данных и вне допустимого диапазона
    valid = (dists_m > config.data.min_range) & (dists_m <
config.data.max_range)

    x = dists_m[valid] * np.cos(angles_rad[valid])
    y = dists_m[valid] * np.sin(angles_rad[valid])
    z = np.zeros(valid.sum())

    return np.column_stack([x, y, z])

while True:
```

Продолжение листинга В.3

```
data, _ = sock.recvfrom(PACKET_SIZE)
telem = parse_telemetry(data)
if telem is None:
    continue

# Преобразуем данные лидара в облако точек
pointcloud = lidar_to_pointcloud(telem["lidar"])

if len(pointcloud) < 10:
    # Слишком мало точек — скан пропускается
    continue

# Передаём скан в KISS-ICP
# register_frame возвращает матрицу позы 4x4 (SE3)
pose = pipeline.register_frame(pointcloud)

# Извлекаем позицию и ориентацию из матрицы позы
x_m = pose[0, 3]
y_m = pose[1, 3]
yaw_rad = np.arctan2(pose[1, 0], pose[0, 0])
yaw_deg = np.rad2deg(yaw_rad)

# Переводим в мм для совместимости с остальным API
x_mm = x_m * 1000.0
y_mm = y_m * 1000.0

print(f"SLAM положение: x={x_mm:.1f} мм y={y_mm:.1f} мм
yaw={yaw_deg:.1f}°")
```


ПРИЛОЖЕНИЕ Г

Копии листов графической части

Листов 6

ВКРБ включает в себя следующий перечень графического материала:

- 1) схема функциональная программной части системы автопилотирования;
- 2) схема функциональная макета автомобиля;
- 3) схема принципиальная макета автомобиля;
- 4) фотографии макета системы;
- 5) формы веб-интерфейса;
- 6) результаты анализа систем автопилотирования.

Копии верны

Руководитель

(Подпись, дата)

Е.Ю. Оболенская
(И.О. Фамилия)

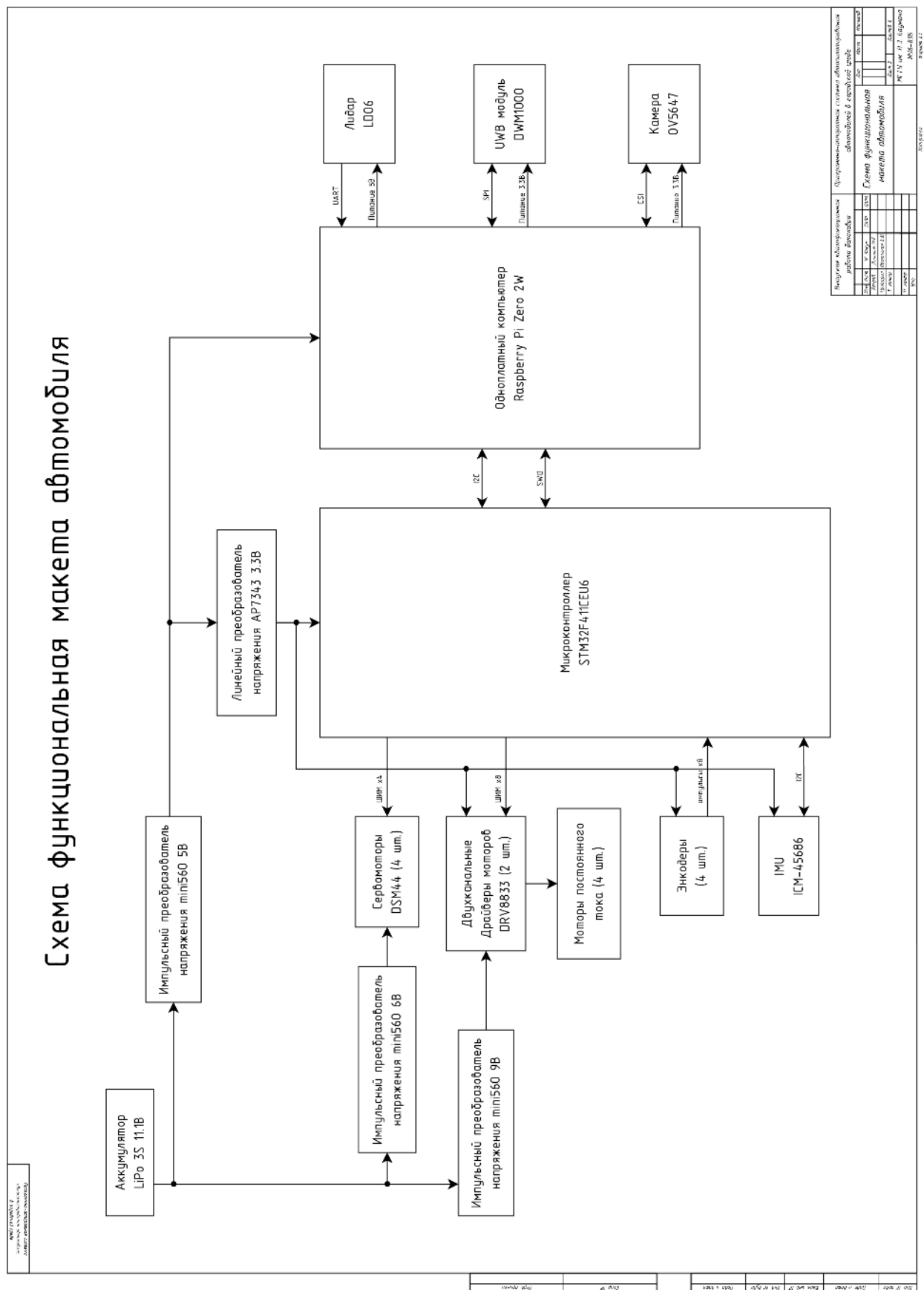


Рисунок Г.2 – Схема функциональная макета автомобиля

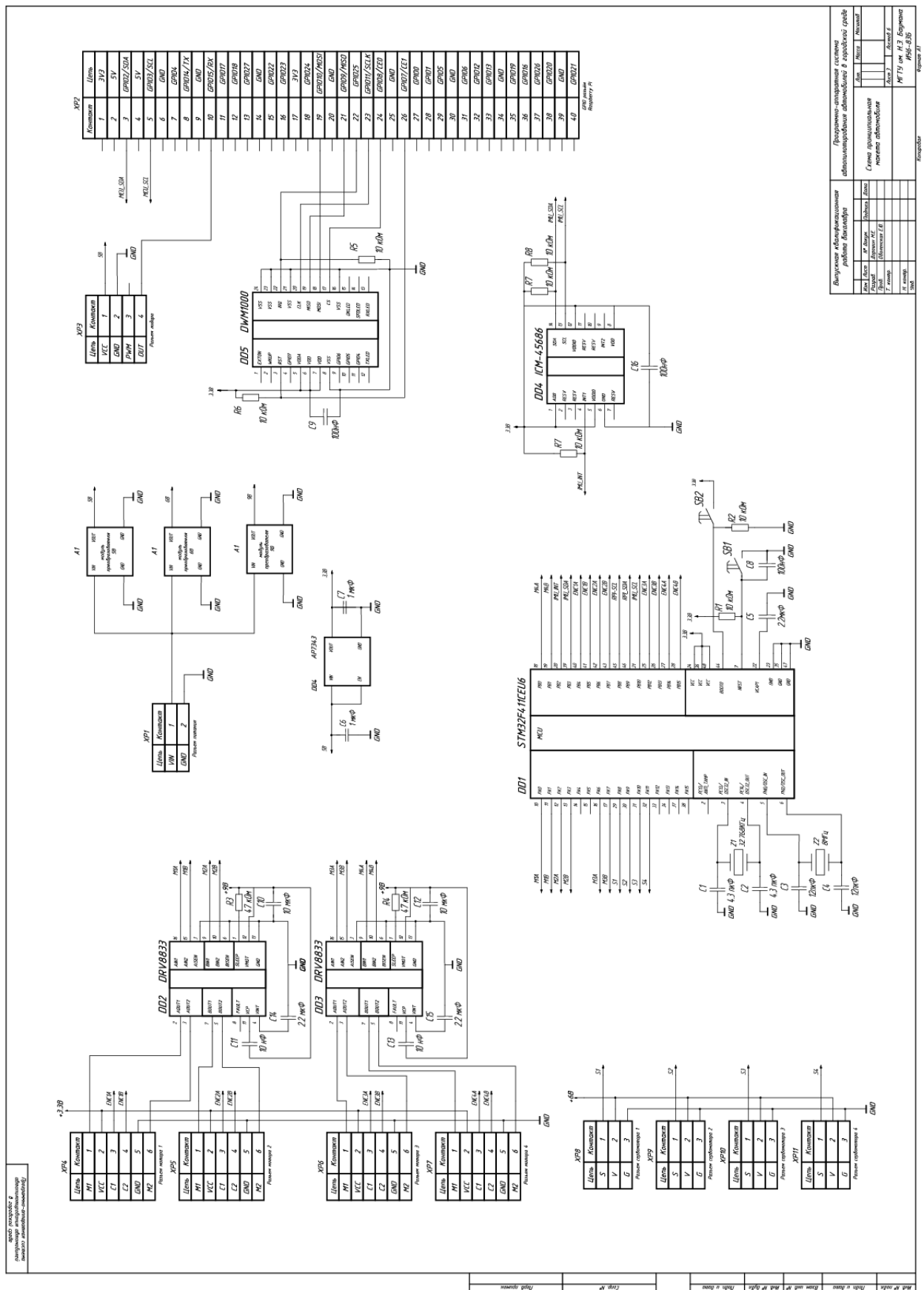
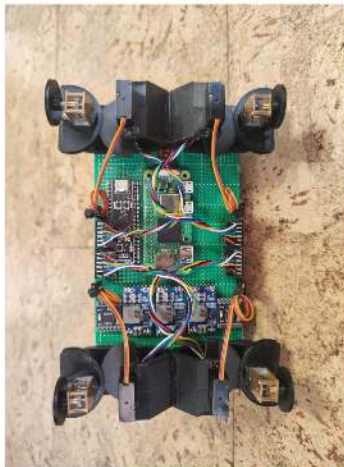


Рисунок Г.3 – Схема принципиальная макета автомобиля



A photograph of a large, white, 3D-printed model of a city street layout, featuring a central intersection and surrounding blocks, placed on a wooden floor. The model is composed of numerous white, rectangular blocks of varying sizes, arranged to represent buildings and street corners. Black lines are drawn on the white surface to delineate the streets and pathways. The model is positioned on a light-colored wooden floor with a visible grain pattern. In the background, a dark wooden cabinet or desk is partially visible.



Светофор + UWB маркер



Среды

Макет города

[illegible]

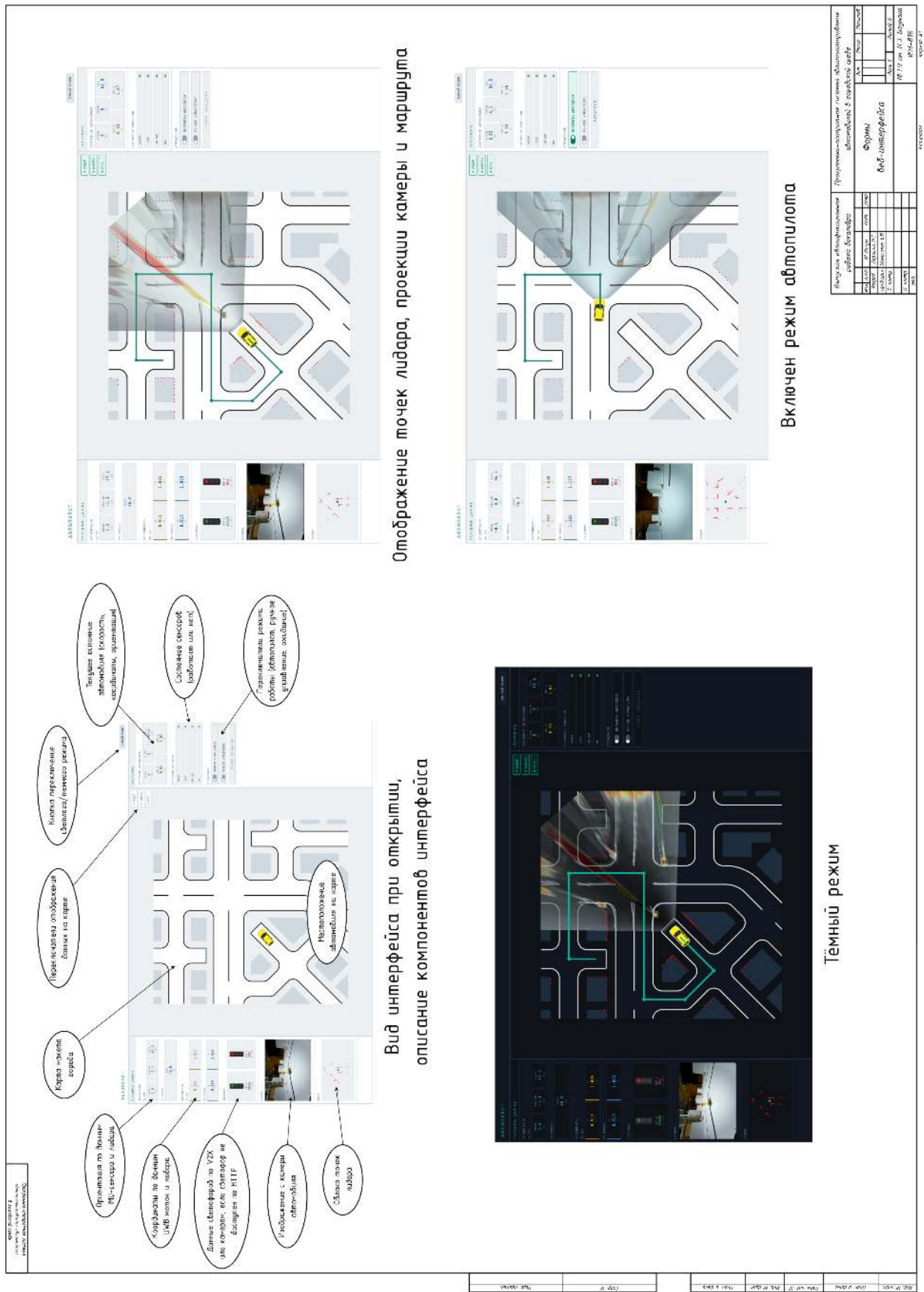


Рисунок Г.5 – Формы веб-интерфейса

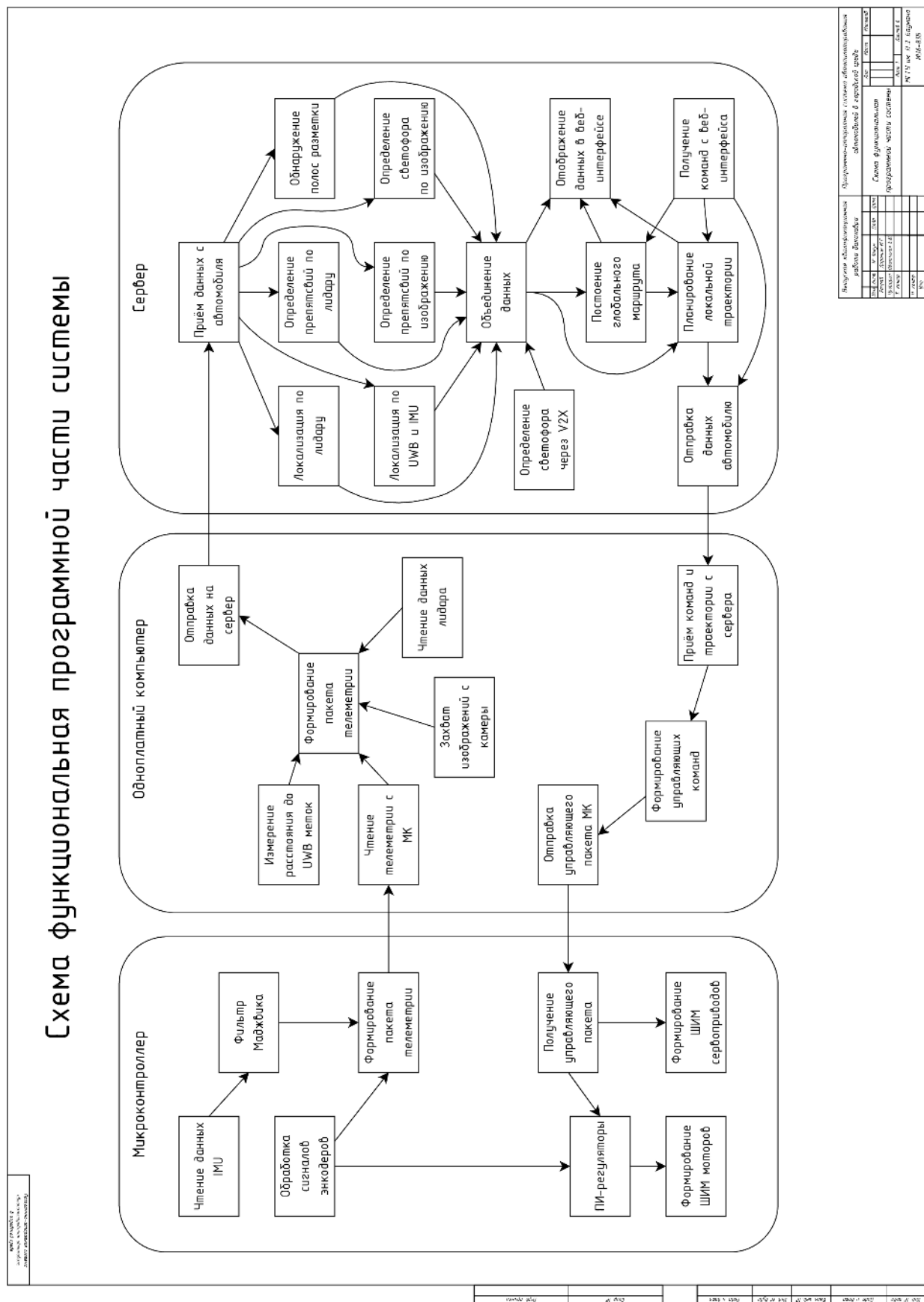


Рисунок Г.6 – Схема функциональная программной части